Machine Learning in Engineering Science

A Colab-first course reader

Chi-Hua Jonny Yu

Fall 2026

How to use this reader

This is the Fall 2026 course reader for *Machine Learning in Engineering Science*. It replaces the 2025 model-by-model textbook structure with a learning sequence that follows the course calendar. The reader is not a catalogue of architectures. Its central question is: *what evidence is sufficient to support an engineering decision?*

Each teaching week combines about 90 minutes of lecture with 90 minutes of practice. For Week 7 these modes are distributed across three independently usable 60-minute modules: sequential data and LSTM, attention, then Transformer architecture and experiments. The website provides the matching notebooks and slide decks. This source keeps the narrative, equations, design rationale, and references in version-controlled LaTeX.

Learning outcomes

By the end of the course, students should be able to:

- formulate an ML problem with a target, features, split, metric, and decision context;
- implement and evaluate a reproducible baseline before seeking a more complex model;
- identify data leakage, overfitting, uncertainty, and failure cases;
- explain when an LLM, retrieval component, tool call, or agent loop is justified; and
- use AI coding assistance while retaining responsibility for tests, evidence, and rollback.

Semester map

Weeks	Course focus	Reader unit and tangible output
1–3	Data workflows and supervised learning	Chapters 1 and 2; a question, a table of features, and a baseline model.
5–6	Evaluation and representation	Chapters 3 and 4; a defensible split, metric, and representation.
7	Sequences, LSTM, attention, and Transformers	Chapters 5 and 6; three one-hour modules with reproducible experiments.
9–10	Reliability and project studio	Chapter 7; a reproducible midterm ML workflow.
11	Midterm assessment	Project demonstration, written examination, and oral examination.
12–14	LLMs, retrieval, tools, and agent design	Chapters 8 to 10; an agent plan with explicit tools and failure cases.
15–17	Vibe coding, tests, and final studio	Chapter 11; an evaluated engineering workflow and final demo.
18	Final assessment	Final project demonstration, written examination, and oral examination.

What is assessed

Labs are active classwork and do not create a weekly graded workload. There are four milestone-based items called *Homework*: an ML problem brief, a reproducible ML workflow, an agent plan, and demo-readiness evidence. The midterm and final each combine a project with written and oral assessment.

A note on AI tools

The course permits free or basic-credit AI tools for drafting code, explanations, and debugging. A generated answer is never sufficient evidence. Submissions must show the model or system used, the prompt or change request when relevant, the tests run, the observed result, and the student's explanation of limits. No project requires a paid API, a dedicated GPU, or a competition ranking.

Contents

How to use this reader i

I Machine Learning Foundations 1

1 Data, Questions, and the ML Workflow 2

1.1 From an engineering question to a learning task . . . 2

1.2 A reproducible first pass . . . 2

1.3 Data contracts . . . 3

2 Supervised Learning: Baselines Before Complexity 4

2.1 Classification and regression . . . 4

2.2 Split before learning from the data . . . 4

2.3 Algorithm choice . . . 5

3 Evaluation, Validation, and Generalization 6

3.1 Metrics carry assumptions . . . 6

3.2 Generalization and the bias–variance trade-off . . . 6

3.3 Validation discipline . . . 7

4 Unsupervised Learning and Representation 8

4.1 Clustering as a hypothesis generator . . . 8

4.2 Representation matters . . . 8

5 Neural Networks and Deep Learning Foundations 10

5.1 A compact mathematical view . . . 10

5.2 Training is an experiment . . . 10

6 Sequential Data, LSTM, Attention, and Transformers 12

6.1 Learning outcomes and evidence . . . 12
6.2 Hour 1: ordered observations and recurrent memory . . . 12
6.2.1 Order, sampling, and the prediction task . . . 12
6.2.2 Chronological splits and available information . . . 13
6.2.3 Baselines and error interpretation . . . 13
6.2.4 RNN state and learning through time . . . 13
6.2.5 The complete LSTM mechanism . . . 14
6.2.6 Implementation, selection, and reproducibility . . . 14
6.3 Hour 2: why attention is useful and how it works . . . 14
6.3.1 The fixed-context bottleneck . . . 14
6.3.2 Queries, keys, values, and shapes . . . 15
6.3.3 An attention calculation with no hidden steps . . . 15
6.3.4 Self-attention, cross-attention, and masks . . . 16
6.3.5 Benefits, costs, and interpretation . . . 16
6.4 Hour 3: assemble and test a Transformer . . . 16
6.4.1 Tokens and position . . . 16
6.4.2 Heads, nonlinear transformations, and residual paths . . . 16
6.4.3 Objective, baseline, and generation . . . 17
6.4.4 Behavioral checks and limits . . . 17
6.5 Completion and explain-back . . . 17
6.6 Primary sources . . . 18

7 Reliability, Error Analysis, and the Midterm Workflow 19
7.1 Week 9: find, explain, change, preserve . . . 19
7.1.1 The four questions . . . 19
7.1.2 Use the notebook independently . . . 20
7.2 Task 1: five errors before a model library . . . 21
7.3 Task 2: the mixture can change the score . . . 23
7.4 Task 3: one transparent main experiment . . . 24
7.4.1 Declare the candidates before inspecting test outcomes . . . 24
7.4.2 A measured reference, not a score to copy . . . 24
7.5 Task 4: make a fair visual comparison . . . 25
7.6 Task 5: input change with an unchanged rule . . . 26
7.7 Task 6: unchanged inputs, changed target relationship . . . 27

7.8 Task 7: check an explanation of the fitted function . . . 28
7.9 Task 8: CSV readback and an honest handover . . . 29
7.9.1 Export and explain-back . . . 29
7.9.2 Troubleshooting . . . 29
7.9.3 Summary . . . 29
7.10 Supplementary reasoning: optional, not extra core tasks . . . 30
7.10.1 Conditional risk and weighted groups . . . 30
7.10.2 Why the population linear approximation misses the square . . . 30
7.10.3 Recovering original-coordinate coefficients . . . 30
7.10.4 Optional A: encoding check . . . 30
7.10.5 Optional B: fixed-model bootstrap . . . 30
7.11 Primary references . . . 31
7.12 Week 10: Reproducible ML Workflow Studio . . . 31
7.12.1 Computational replay has a bounded meaning . . . 31
7.13 Stable identity, not a seed alone . . . 31
7.14 The packaged regression fixture and its limits . . . 32
7.14.1 An executable interface . . . 32
7.15 Baseline, selection and held-out evidence . . . 32
7.16 A checkable function explanation . . . 34
7.17 What belongs in the handover . . . 34
7.17.1 Integrity is not authentication . . . 35
7.18 Replay outside notebook memory . . . 35
7.19 Test deliberate failures on copies . . . 36
7.20 Eight core exercises and two optional challenges . . . 36
7.20.1 Student responses and evidence export . . . 37
7.21 Instructor execution and limits . . . 38
7.22 Summary and examination bridge . . . 38
7.23 Primary-source reading . . . 38
7.24 Weeks 10–11: workflow and examination preparation . . . 38

II Language Models, Agents, and Engineering Workflows 39

8 LLMs, bounded agents, and vibe coding 40

8.1 H1: From next-token prediction to tool proposals . . . 40
8.1.1 Language is an interface, not an oracle . . . 40
8.1.2 A token is not necessarily a word . . . 40
8.1.3 Autoregression factorizes a sequence . . . 41
8.1.4 Cross-entropy penalizes surprise . . . 41
8.1.5 Attention selects context-dependent mixtures . . . 41
8.1.6 Pretraining and post-training differ . . . 41
8.1.7 Sampling changes a distribution . . . 42
8.1.8 A tiny language model is inspectable . . . 42
8.1.9 Prediction before sampling creates evidence . . . 42
8.1.10 Summary: separate probability and proof . . . 42
8.2 H2: Structured outputs and bounded tool execution . . . 42
8.2.1 The tool boundary is executable code . . . 42
8.2.2 JSON syntax is only the first gate . . . 43
8.2.3 Use exact fields and explicit types . . . 43
8.2.4 No natural-language sentence grants tools . . . 43
8.2.5 Numerical results need their own checks . . . 43
8.2.6 A finite budget belongs to the harness . . . 44
8.2.7 Trace observable events, not hidden thoughts . . . 44
8.2.8 An LLM adapter is optional, not required . . . 44
8.2.9 Vibe coding needs acceptance criteria . . . 44
8.2.10 Summary: the model never owns permissions . . . 45
8.3 H3: Tool-contract studio: test, explain, export . . . 45
8.3.1 Build a proposal-to-evidence artifact . . . 45
8.3.2 Task 1: read a learned distribution . . . 45
8.3.3 Task 2: compare sampling controls . . . 45
8.3.4 Task 3: reject the unknown capability . . . 45
8.3.5 Task 4: classify malformed readings . . . 46
8.3.6 Task 5: stop a repeated proposal loop . . . 46
8.3.7 Task 6: repair syntax without changing intent . . . 46
8.3.8 Task 7: expose arithmetic with a fixture . . . 46
8.3.9 Task 8: hand over the evidence ZIP . . . 47
8.3.10 Summary: extend only a declared boundary . . . 47

8.4 Mathematical reference . . . 47
8.5 Eight core practice tasks . . . 47
8.5.1 Task 1: Inspect a learned next-character distribution . . . 47
8.5.2 Task 2: Change temperature without changing the model . . . 48
8.5.3 Task 3: Reject an unknown tool before execution . . . 48
8.5.4 Task 4: Probe malformed numeric inputs . . . 48
8.5.5 Task 5: Exhaust the attempt budget . . . 48
8.5.6 Task 6: Repair syntax while preserving the boundary . . . 48
8.5.7 Task 7: Independently check arithmetic . . . 48
8.5.8 Task 8: Audit what the exported record can support . . . 49
8.6 Evidence handover and summary . . . 49
8.7 Primary references . . . 49

9 Retrieval, source evidence, and structured knowledge 50
9.1 H1: Represent documents and retrieve evidence . . . 50
9.1.1 Retrieval selects evidence, not truth . . . 50
9.1.2 Document identity must survive chunking . . . 50
9.1.3 Tokenization defines lexical overlap . . . 51
9.1.4 IDF discounts corpus-common terms . . . 51
9.1.5 TF-IDF is a fitted representation . . . 51
9.1.6 Cosine compares directions . . . 51
9.1.7 Top-k is a selection rule . . . 52
9.1.8 Thresholds trade coverage and abstention . . . 52
9.1.9 Chunking can remove required context . . . 52
9.1.10 Summary: freeze the representation . . . 52
9.2 H2: Measure retrieval and check claim support . . . 53
9.2.1 Two evaluation layers answer different questions . . . 53
9.2.2 Recall at k needs a relevance set . . . 53
9.2.3 Reciprocal rank rewards the first hit . . . 53
9.2.4 An evidence quote has a narrow role . . . 53
9.2.5 Task 3: score fixed retrieval questions . . . 53
9.2.6 Task 4: inspect false support . . . 54
9.2.7 Task 5: change k under a fixed index . . . 54
9.2.8 Treat source instructions as data . . . 54

9.2.9 Structured knowledge needs provenance too . . . 54
9.2.10 Summary: evidence retrieval precedes explanation . . . 55
9.3 H3: Retrieval studio: diagnose and package failures . . . 55
9.3.1 Build an evidence-first retrieval record . . . 55
9.3.2 Task 1: distinguish answerable and unknown . . . 55
9.3.3 Task 2: explain one weighted term . . . 55
9.3.4 Task 3: compute macro retrieval recall . . . 56
9.3.5 Task 4: find a citation failure . . . 56
9.3.6 Task 5: vary k without tuning on the final set . . . 56
9.3.7 Task 6: add a genuinely independent query . . . 56
9.3.8 Task 7: separate tool results from passages . . . 56
9.3.9 Task 8: export source-bounded evidence . . . 57
9.3.10 Summary: ask what the source entails . . . 57
9.4 Mathematical reference . . . 57
9.5 Eight core practice tasks . . . 57
9.5.1 Task 1: Compare relevant evidence and abstention . . . 57
9.5.2 Task 2: Inspect the fitted lexical representation . . . 58
9.5.3 Task 3: Evaluate fixed held-out relevance labels . . . 58
9.5.4 Task 4: Separate citation structure from claim support . . . 58
9.5.5 Task 5: Change k on development questions only . . . 58
9.5.6 Task 6: Add one independently labeled diagnostic query . . . 58
9.5.7 Task 7: Combine a numeric result and a source rule . . . 58
9.5.8 Task 8: Preserve corpus identity and one failure . . . 59
9.6 Evidence handover and summary . . . 59
9.7 Primary references . . . 59

10 Agent testing, failure cases, and oversight 60
10.1 H1: State, budgets, and trust boundaries . . . 60
10.1.1 An agent is a controlled loop . . . 60
10.1.2 Workflow and agent are not synonyms . . . 60
10.1.3 Represent state before writing actions . . . 61
10.1.4 Budgets need an accounting unit . . . 61
10.1.5 Validate before dispatch . . . 61
10.1.6 Use typed errors with narrow recovery . . . 61

10.1.7 Untrusted text cannot amend policy . . . 62
10.1.8 Approval must name the exact action . . . 62
10.1.9 Consumption prevents approval replay . . . 62
10.1.10 Summary: make unsafe transitions impossible . . . 62
10.2 H2: Failure cases and human approval tests . . . 63
10.2.1 A safety claim needs a negative test . . . 63
10.2.2 Test status and side effects together . . . 63
10.2.3 Task 4: probe schema edge cases . . . 63
10.2.4 Task 5: exhaust the attempt cap . . . 63
10.2.5 Task 6: bind approval to arguments . . . 63
10.2.6 Simulated timeout is not process isolation . . . 64
10.2.7 Retry only a declared transient failure . . . 64
10.2.8 Audit traces are bounded evidence . . . 64
10.2.9 Human review must offer a real choice . . . 64
10.2.10 Summary: safety and usefulness are separate . . . 65
10.3 H3: Oversight studio: attack the boundary . . . 65
10.3.1 Probe the harness without real side effects . . . 65
10.3.2 Task 1: accept a legitimate operation . . . 65
10.3.3 Task 2: keep malicious text powerless . . . 65
10.3.4 Task 3: prove one-use approval . . . 66
10.3.5 Task 4: differentiate malformed inputs . . . 66
10.3.6 Task 5: inspect the terminal state . . . 66
10.3.7 Task 6: modify an approved request . . . 66
10.3.8 Task 7: reconstruct the first failure . . . 66
10.3.9 Task 8: preserve a negative result . . . 67
10.3.10 Summary: extend one threat model at a time . . . 67
10.4 Invariant reference . . . 67
10.5 Eight core practice tasks . . . 67
10.5.1 Task 1: Accept a valid allowed calculation . . . 67
10.5.2 Task 2: Keep hostile source text outside authority . . . 68
10.5.3 Task 3: Use approval once and reject replay . . . 68
10.5.4 Task 4: Probe data boundaries and side effects . . . 68
10.5.5 Task 5: Prove a terminal budget state . . . 68

10.5.6 Task 6: Reject changed or expired approvals . . . 68
10.5.7 Task 7: Read the first failed invariant . . . 68
10.5.8 Task 8: Separate finite-suite evidence and broad claims . . . 69
10.6 Evidence handover and summary . . . 69
10.7 Primary references . . . 69

11 Agent Studio I: build a bounded workflow 70
11.1 H1: Specify an engineering evidence workflow . . . 70
11.1.1 Build a report, not a physical controller . . . 70
11.1.2 A row needs physical meaning . . . 70
11.1.3 Units are part of the input type . . . 71
11.1.4 Summaries answer different questions . . . 71
11.1.5 Missing and nonfinite data require a policy . . . 71
11.1.6 The threshold comes from a source . . . 71
11.1.7 Predeclare behavior for disagreement . . . 72
11.1.8 Separate data errors from review decisions . . . 72
11.1.9 Make the report schema explicit . . . 72
11.1.10 Summary: define meaning before automation . . . 72
11.2 H2: Implement a checked analysis pipeline . . . 72
11.2.1 The workflow is a graph with gates . . . 72
11.2.2 Validate the complete request . . . 73
11.2.3 Conversion needs fixture tests . . . 73
11.2.4 Preserve the threshold comparison policy . . . 73
11.2.5 The sample SD has a declared denominator . . . 73
11.2.6 Trace stage outputs without excessive data . . . 74
11.2.7 Abstention is a valid terminal result . . . 74
11.2.8 A report must distinguish facts and limits . . . 74
11.2.9 Test an AI-generated patch as new code . . . 74
11.2.10 Summary: preserve the reference workflow . . . 74
11.3 H3: Studio I: build, probe, and explain . . . 75
11.3.1 Deliver one complete analysis chain . . . 75
11.3.2 Task 1: classify the contract outcomes . . . 75
11.3.3 Task 2: audit provenance fields . . . 75
11.3.4 Task 3: prove unit equivalence . . . 75

11.3.5 Task 4: explain one spread calculation . . . 76
11.3.6 Task 5: locate a stopped stage . . . 76
11.3.7 Task 6: remove source authority . . . 76
11.3.8 Task 7: diagnose the denominator patch . . . 76
11.3.9 Task 8: package and hand over . . . 77
11.3.10 Summary: expand scope only with new tests . . . 77
11.4 Mathematical reference . . . 77
11.5 Eight core practice tasks . . . 77
11.5.1 Task 1: Predict the four contract outcomes . . . 77
11.5.2 Task 2: Inspect provenance before prose . . . 77
11.5.3 Task 3: Check equivalent Celsius and Fahrenheit requests . . . 78
11.5.4 Task 4: Reconstruct the sample spread . . . 78
11.5.5 Task 5: Stop at the invalid unit boundary . . . 78
11.5.6 Task 6: Abstain without approved source evidence . . . 78
11.5.7 Task 7: Expose a plausible denominator patch . . . 78
11.5.8 Task 8: Save the input, protocol, source and result . . . 78
11.6 Evidence handover and summary . . . 79
11.7 Primary references . . . 79

12 Agent Studio II: evaluation and demo rehearsal 80
12.1 H1: Design an evaluation before the demonstration . . . 80
12.1.1 Evaluate behavior, not presentation polish . . . 80
12.1.2 The case is the unit of evaluation . . . 80
12.1.3 Expected answers must be independent . . . 81
12.1.4 Separate development and held-out cases . . . 81
12.1.5 Status correctness and numeric accuracy differ . . . 81
12.1.6 Numerical tolerance is a rule, not a repair . . . 81
12.1.7 An error budget is not a safety excuse . . . 82
12.1.8 The baseline makes added complexity accountable . . . 82
12.1.9 Predeclare a release rule . . . 82
12.1.10 Summary: freeze what will be compared . . . 82
12.2 H2: Run the harness and explain candidate failures . . . 82
12.2.1 One harness must judge both candidates . . . 82
12.2.2 Return structured failures, not silent skips . . . 83

12.2.3 Task 3: inspect the per-case table . . . 83
12.2.4 A zero-mean case can conceal a bug . . . 83
12.2.5 Measure the declared subgroup outcomes . . . 83
12.2.6 Fixed-case uncertainty is not population certainty . . . 84
12.2.7 Task 5: inspect an invalid-input success . . . 84
12.2.8 Task 6: apply the frozen release gate . . . 84
12.2.9 Rehearse evidence before screenshots . . . 84
12.2.10 Summary: explain why a candidate failed . . . 85
12.3 H3: Studio II: evaluate, challenge, rehearse . . . 85
12.3.1 Keep the evaluator independent of the candidate . . . 85
12.3.2 Task 1: predict which fixture exposes n+1 . . . 85
12.3.3 Task 2: freeze the release gate . . . 85
12.3.4 Task 3: inspect one failing row . . . 85
12.3.5 Task 4: explain the hidden zero numerator . . . 86
12.3.6 Task 5: check a required rejection . . . 86
12.3.7 Task 6: defend the release decision . . . 86
12.3.8 Task 7: rehearse the five-minute narrative . . . 86
12.3.9 Task 8: export an inspectable evaluation . . . 87
12.3.10 Summary: improve tests before adding autonomy . . . 87
12.4 Evaluation reference . . . 87
12.5 Eight core practice tasks . . . 87
12.5.1 Task 1: Predict which fixture exposes the denominator . . . 87
12.5.2 Task 2: Freeze the release gate and candidate identities . . . 87
12.5.3 Task 3: Run the same held-out harness for both candidates . . . 88
12.5.4 Task 4: Explain why the zero case hid a defect . . . 88
12.5.5 Task 5: Inspect required rejections separately . . . 88
12.5.6 Task 6: Apply the unchanged classroom release gate . . . 88
12.5.7 Task 7: Rehearse a five-minute evidence narrative . . . 88
12.5.8 Task 8: Export rows, not only averages . . . 88
12.6 Evidence handover and summary . . . 89
12.7 Primary references . . . 89

13 Project integration and oral-exam preparation 90
13.1 H1: Integrate the final evidence contract . . . 90

13.1.1 Integration means a reconstructable result . . . 90
13.1.2 Scope the final claim before packaging . . . 90
13.1.3 The manifest names required payloads . . . 91
13.1.4 Use canonical JSON deliberately . . . 91
13.1.5 Source is safer to inspect than opaque objects . . . 91
13.1.6 A new process removes hidden globals . . . 91
13.1.7 Replay compares computations, not outcomes . . . 92
13.1.8 Align identities before values . . . 92
13.1.9 Record the actual environment . . . 92
13.1.10 Summary: package the claim with its limits . . . 92
13.2 H2: Replay, integrity checks, and oral defense . . . 93
13.2.1 The ZIP must contain everything it promises . . . 93
13.2.2 Task 4: extract into a fresh directory . . . 93
13.2.3 Task 5: replay from saved files . . . 93
13.2.4 Task 6: compare an aligned result vector . . . 93
13.2.5 Tamper detection is not authenticity . . . 94
13.2.6 Explain the mechanism in a short defense . . . 94
13.2.7 Budget the five-minute demonstration . . . 94
13.2.8 Written and oral assessment remain separate . . . 94
13.2.9 Peer handover is an adversarial usability test . . . 94
13.2.10 Summary: preserve the observed state . . . 95
13.3 H3: Final integration studio and explain-back . . . 95
13.3.1 Finish a rerunnable bounded project record . . . 95
13.3.2 Task 1: locate every hidden dependency . . . 95
13.3.3 Task 2: check case identity and ordering . . . 95
13.3.4 Task 3: state the actual environment . . . 96
13.3.5 Task 4: inspect the extracted package . . . 96
13.3.6 Task 5: require actual process completion . . . 96
13.3.7 Task 6: explain computational agreement . . . 96
13.3.8 Task 7: detect copied-file tampering . . . 97
13.3.9 Task 8: defend claim and limitation . . . 97
13.3.10 Summary: prepare, do not replace, the final exam . . . 97
13.4 Replay reference . . . 97

13.5 Eight core practice tasks . . . 97
13.5.1 Task 1: Find notebook-only dependencies . . . 97
13.5.2 Task 2: Check identity before numeric values . . . 98
13.5.3 Task 3: Record the environment that actually ran . . . 98
13.5.4 Task 4: Extract a whitelisted handover ZIP . . . 98
13.5.5 Task 5: Require a completed new process . . . 98
13.5.6 Task 6: Compare aligned numeric outputs . . . 98
13.5.7 Task 7: Detect tampering on a copy . . . 98
13.5.8 Task 8: Create a defensible handover card . . . 99
13.6 Evidence handover and summary . . . 99
13.7 Primary references . . . 99

A Colab Conventions and Homework Milestones **100**
A.1 Colab conventions . . . 100
A.2 Homework milestones . . . 100
A.3 Submission rule . . . 101
A.4 Runtime-aware Lab design . . . 101

B Extension Studios: Deep Learning Architectures **102**
B.1 Convolutional neural networks . . . 102
B.2 Sequential models . . . 102
B.3 Generative models . . . 102

Part I

Machine Learning Foundations

Chapter 1

Data, Questions, and the ML Workflow

Weeks 1–2. A useful ML system begins before model selection. It begins with a decision, a measurable target, a representation of the available evidence, and a procedure that can be repeated. This chapter establishes the working vocabulary used throughout the course.

Learning outcomes

- Distinguish a decision problem from an ML task.
- Identify an observation, feature vector x, target y, and prediction $\hat{y}$.
- Create a small, inspectable data workflow in Python.
- State what a plot or summary statistic does *not* establish.

1.1 From an engineering question to a learning task

An engineering question such as "Which process condition is likely to produce a defect?" must be made operational. Define the unit of observation, the time at which the prediction is made, the target, the available features, and the decision that follows. A classification task predicts a discrete label; a regression task predicts a continuous value. Neither label is meaningful without a stated measurement process and error cost.

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ denote observed examples. A learning algorithm selects a function f_θ so that $\hat{y} = f_\theta(x)$ is useful on data not used to choose θ. The course treats this as an evidence problem: the data split and evaluation protocol are part of the model, not afterthoughts [7].

1.2 A reproducible first pass

Before fitting a model, inspect shape, column meaning, units, missingness, and plausible ranges. The following pattern is deliberately small enough to read line by line.

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(2026)
df = pd.DataFrame({
    "temperature_c": rng.normal(72, 5, 40),
    "pressure_kpa": rng.normal(210, 18, 40),
})
df["defect_flag"] = (
    (df["temperature_c"] > 75) & (df["pressure_kpa"] > 220)
).astype(int)

print(df.shape)
print(df.describe())
plt.scatter(df["temperature_c"], df["pressure_kpa"],
            c=df["defect_flag"], cmap="coolwarm")
plt.xlabel("Temperature (C)")
plt.ylabel("Pressure (kPa)")
plt.show()
```

A coloured scatter plot can reveal a possible association. It does not prove causality, establish a production threshold, or show that a future batch will behave similarly. The lab asks students to write both the supported observation and a limitation.

Colab Lab

Week 1: Open in Google Colab. Inspect a measurement table, compute descriptive statistics, create one figure, and write an evidence statement. The target is not yet a high-scoring model; it is a clear workflow that another person can reproduce.

1.3 Data contracts

A data contract records the feature names, units, allowable values, target definition, split rule, and known missingness. It is lightweight documentation with a strong engineering benefit: it prevents silent changes from being mistaken for model improvement. For every course notebook, include a first cell that reports package versions, a random seed, data provenance, and the shape of the loaded table.

Checkpoint

For a proposed predictor of fatigue life, name: (1) one observation, (2) three candidate features, (3) the target and its unit, (4) a decision that uses the prediction, and (5) one feature that would be unavailable at prediction time. Explain why the last feature must be excluded.

Chapter 2

Supervised Learning: Baselines Before Complexity

Weeks 2–3. Supervised learning uses labelled examples to learn a mapping from features to a target. The practical discipline is to establish a transparent baseline, then compare alternatives under the same split and metric.

Learning outcomes

- Separate feature engineering from target definition.
- Construct train and test partitions without leakage.
- Fit a baseline regression or classification pipeline.
- Interpret a confusion matrix and a residual plot in context.

2.1 Classification and regression

For binary classification, the model estimates a score or probability $p_\theta(y = 1 \mid x)$. A threshold turns the score into an action. The threshold should follow the cost of false positives and false negatives, not a default convention. For regression, a common starting loss is mean squared error,

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2,$$

but an error in megapascals, minutes, or dollars must be interpreted in its application context.

2.2 Split before learning from the data

Data leakage occurs when information from the test set changes feature construction, scaling, imputation, model selection, or threshold choice. A pipeline makes the correct order explicit: fit transformations on training data only, then apply the learned transformations to held-out data.

```python
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

wine = load_wine(as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(
    wine.data, wine.target, test_size=0.25,
    random_state=2026, stratify=wine.target,
)
model = Pipeline([
    ("scale", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=2000)),
])
model.fit(X_train, y_train)
print(classification_report(y_test, model.predict(X_test)))
```

A logistic-regression pipeline is not "simple" in the sense of being disposable. It establishes a reproducible reference point. A more complex model earns its place only when the evaluation protocol demonstrates an improvement relevant to the decision.

2.3 Algorithm choice

Decision trees expose rule-like splits; nearest-neighbour methods compare local examples; support vector machines seek a separating margin; ensembles combine diverse predictors. These methods remain useful when the dataset is structured, moderate in size, and the evaluation contract is clear [1, 11]. The course uses them to develop comparison discipline, rather than to encourage architecture shopping.

Colab Lab

Week 2: Open in Google Colab. Build a leakage-safe baseline classifier, report accuracy and a confusion matrix, then change one decision and document its effect. Your explanation is more important than the numerical score alone.

Checkpoint

A feature is calculated using the mean of every row in the dataset before the train/test split. Is this leakage? Describe the corrected order of operations and explain why it matters even if the measured score changes only slightly.

Chapter 3

Evaluation, Validation, and Generalization

Week 5. A score is evidence only when it was produced by a protocol that matches the intended use. This chapter makes error analysis, validation, and model selection explicit.

Learning outcomes

- Select metrics that expose the errors that matter.
- Explain underfitting, overfitting, and the bias–variance trade-off.
- Use cross-validation without contaminating the final test set.
- Turn model errors into a next experimental question.

3.1 Metrics carry assumptions

Accuracy is the fraction of correct predictions, but it can hide failures when classes are imbalanced. Precision asks whether positive predictions are credible; recall asks whether actual positives are found. For a binary classifier,

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}.$$

Choose the metric before inspecting the final score. A defect-screening model might favour recall, while an expensive destructive test may require higher precision.

3.2 Generalization and the bias–variance trade-off

Training performance can improve while test performance degrades. A model with too little capacity underfits; a model tuned too aggressively to a small training set overfits. The remedy is not always a larger model. Better measurements, fewer unstable features, regularization, or a clearer task definition may be more valuable.

3.3 Validation discipline

Use training data to fit candidate pipelines, validation or cross-validation to select among them, and a final test set for one honest estimate. Record the split seed, preprocessing, metric, and selection rule. Do not repeat test-set evaluation until it becomes an informal validation loop.

Colab Lab

Week 5: compare two pipelines using the same split and metric. Make a plot of training and validation behaviour, then write a short error analysis: which cases fail, what shared property might explain the failure, and what additional data would reduce uncertainty?

Checkpoint

A classifier has 98% accuracy but identifies only 2 of 40 safety-critical failures. Which additional metric would you inspect first, and what decision trade-off does it reveal?

Chapter 4

Unsupervised Learning and Representation

Week 6. Unsupervised methods organize data without a labelled target. They are useful for exploration, anomaly investigation, compression, and finding candidate subpopulations. They do not automatically create ground truth.

Learning outcomes

- Explain the difference between clustering and classification.
- Standardize features appropriately before distance-based analysis.
- Use a low-dimensional view as an exploratory tool rather than a final conclusion.
- State how a discovered cluster would be checked against domain knowledge.

4.1 Clustering as a hypothesis generator

Methods such as k-means partition observations by geometric similarity. The algorithm returns groups even when no meaningful groups exist. Therefore a cluster label should be treated as a hypothesis to investigate with process knowledge, additional measurements, and sensitivity analysis.

4.2 Representation matters

Distances depend on feature scale and representation. A temperature difference of 10 degrees and a pressure difference of 10 kPa are not inherently comparable. Document scaling, missing-value handling, and the variables intentionally excluded from the analysis.

Colab Lab

Week 6: standardize a multivariate dataset, compare two values of k, and inspect the stability of the result. Identify one observation that would require domain review rather than automatic labelling.

Chapter 5

Neural Networks and Deep Learning Foundations

Week 7. Deep learning learns representations through composed differentiable functions. The course focuses on the training loop, loss surface, and diagnostic signals needed to use a neural network responsibly.

Learning outcomes

- Describe a forward pass, loss, gradient, and parameter update.
- Relate activation functions and network capacity to optimization behaviour.
- Read a training curve for divergence, underfitting, and overfitting.
- Compare a neural baseline with a classical baseline under the same protocol.

5.1 A compact mathematical view

For one layer, $h = \phi(Wx + b)$, where W and b are learned parameters and ϕ is a nonlinear activation. A loss $\mathcal{L}(y, \hat{y})$ measures discrepancy. Backpropagation applies the chain rule to compute $\nabla_\theta \mathcal{L}$ efficiently [9]. Gradient-based optimization then updates parameters, for example $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$.

5.2 Training is an experiment

Fix seeds where possible, retain the split, log loss and metric traces, and compare a neural network to a simpler baseline. A training curve is not a certificate of generalization. It is an instrument that helps identify whether the next action is to inspect data, change capacity, regularize, or stop.

Colab Lab

Week 7: train a small PyTorch network on a compact dataset using CPU or optional Colab acceleration. Keep the network deliberately small. Compare it to a classical baseline and write what the extra capacity changed.

Chapter 6

Sequential Data, LSTM, Attention, and Transformers

Week 7: Monday 19 October 2026, 14:00–17:00.

The three one-hour modules follow one question: how can a learned model use earlier information to make a defensible prediction? Hour 1 develops recurrent memory, Hour 2 develops query-dependent retrieval, and Hour 3 assembles and tests a causal Transformer. About 90 minutes of explanation and worked reasoning are combined with 90 minutes of prediction, guided computation, experimentation, and explain-back across the three hours.

6.1 Learning outcomes and evidence

Students should be able to define a forecast origin and horizon; construct windows without future leakage; trace every LSTM gate; compute attention scores, weights, and a context vector; distinguish self-attention from cross-attention; explain positional information, multi-head attention, residuals, and feed-forward layers; and test a decoder's causal behavior. The three independent CPU notebooks compare an LSTM against baselines, verify attention arithmetic, and train and inspect a small Transformer. All data are synthetic or self-contained. No paid API or GPU is required. These are class exercises, not additional weekly graded assignments.

6.2 Hour 1: ordered observations and recurrent memory

6.2.1 Order, sampling, and the prediction task

A sequence is an ordered collection $x_1, \ldots, x_T$ in which order is part of the problem definition. Sensor samples, process events, and language tokens are examples. $(1, 2, 4, 8)$ and $(8, 1, 4, 2)$ have the same multiset and mean but different transitions. A representation that discards order cannot recover those transitions from the values alone. Equally spaced samples and irregular events also require different sampling assumptions. A sequence model does not automatically infer omitted time intervals.

In the example, the forecast origin is immediately before observing x_t. With window length L,

$$X_t = (x_{t-L}, \ldots, x_{t-1}), \qquad y_t = x_t.$$

The Python slice `signal[t-L:t]` excludes index t. Every input index must be strictly less than the target index. This is a one-sample horizon. A fixed-origin multi-step forecast cannot use later actual values that would not yet have been observed.

With B histories, T time steps, and F features at each step, batch-first input has shape (B, T, F). Many-to-one predicts one value per history; many-to-many predicts outputs at multiple positions. Batch size is not sequence length, and an ID is not necessarily a physical feature.

6.2.2 Chronological splits and available information

The synthetic sensor uses 1600 samples, window length 24, training targets before index 960, validation targets 960–1279, and test targets 1280 onward. This produces 936 training, 320 validation, and 320 test windows. Target periods are disjoint. A test window may use earlier observed validation or test sensor history. This is explicitly a rolling one-step protocol, not a forecast of all future values from one initial origin.

Normalization learns statistics, so fit it on training observations only:

$$z_t = (x_t - \mu_{\text{train}})/\sigma_{\text{train}}.$$

Global means can leak future distribution information even without labels. Random splits of heavily overlapping windows can produce nearly identical training and test histories. For multiple devices or people, group separation may be needed in addition to temporal separation.

6.2.3 Baselines and error interpretation

Persistence uses $\hat{x}_t = x_{t-1}$ and has no fitted parameters. Ridge regression uses a regularized linear map of the flattened history window. Select regularization on validation, then compare the same test targets:

$$\text{MAE} = \frac{1}{N}\sum_n |\hat{y}_n - y_n|, \quad \text{RMSE} = \sqrt{\frac{1}{N}\sum_n (\hat{y}_n - y_n)^2}.$$

Report original target units and inspect residuals against time or operating range. RMSE emphasizes large errors. Neither score alone reveals every failure. A linear model can outperform an LSTM on a small, structured signal; report the observed comparison rather than choosing the more complex name.

6.2.4 RNN state and learning through time

A basic recurrent network reuses one update at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b).$$

Unrolling displays repeated applications of shared parameters. The state is a learned summary, not necessarily an interpretable physical variable. Backpropagation through time includes products of

Jacobians:

$$\frac{\partial h_t}{\partial h_{t-k}} = J_t J_{t-1} \cdots J_{t-k+1}.$$

Repeated contractions may shrink gradients, and repeated expansions may amplify them. A scalar example 0.9^{100} illustrates one contraction mechanism but not a complete nonlinear Jacobian product. Gradient clipping limits large gradient magnitude; it does not itself solve vanishing gradients.

6.2.5 The complete LSTM mechanism

An LSTM separates cell memory c_t from hidden output h_t. With $u_t = [h_{t-1}; x_t]$, a standard cell uses

$$f_t = \sigma(W_f u_t + b_f), \qquad i_t = \sigma(W_i u_t + b_i), \tag{6.1}$$

$$g_t = \tanh(W_g u_t + b_g), \qquad o_t = \sigma(W_o u_t + b_o), \tag{6.2}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t). \tag{6.3}$$

Here $\sigma(z) = 1/(1 + e^{-z})$ and $\odot$ is elementwise multiplication. Each gate is a vector of coefficients, generally one per hidden dimension. The forget gate retains old cell content; the input gate scales a signed candidate write; the output gate exposes a filtered view of the updated cell. The additive memory path can improve gradient flow but does not guarantee unlimited retention or eliminate every optimization difficulty. The direct cell-state path contains products of forget gates when their other dependencies are held fixed; the full derivative is more complicated.

Choose illustrative values $c_{t-1} = 0.8$, $f_t = 0.9$, $i_t = 0.2$, $g_t = -0.5$, and $o_t = 0.7$. Then

$$c_t = 0.72 - 0.10 = 0.62, \qquad h_t = 0.7 \tanh(0.62) \approx 0.386.$$

These are hand-chosen teaching values, not training outputs. If $f_t = 1$, new content can still change the cell. Changing o_t changes hidden output without changing this cell update. Predict those effects before using sliders.

6.2.6 Implementation, selection, and reproducibility

With `batch_first=True`, input/output layouts are (B, T, F) and (B, T, H). Ordinary non-projected hidden and cell states retain layout (layers $\times$ directions, B, H). PyTorch concatenates gate chunks in input, forget, candidate, output order. The notebook reconstructs a learned cell and compares it numerically with `nn.LSTM`. Training updates use training data, validation selects a checkpoint, and test is evaluated afterward. Save raw data identity, seed, versions, window, split boundaries, normalization, model state, baselines, and measured errors. A seed alone does not guarantee bitwise agreement across environments.

6.3 Hour 2: why attention is useful and how it works

6.3.1 The fixed-context bottleneck

A final-state encoder compresses a source sequence into one fixed-width context. Different output steps may need different earlier details. Attention makes encoder states addressable and forms a

context for each query. It does not guarantee correct retrieval or unlimited context. Attention can be combined with recurrence: Bahdanau and colleagues used recurrent sequence-to-sequence modeling in a 2014 preprint presented in 2015. The Transformer architecture came later. Distinguish a retrieval mechanism from the entire architecture that uses it.

6.3.2 Queries, keys, values, and shapes

Queries encode a retrieval need. Keys encode matching information, while the corresponding value rows carry content to combine. For N_q queries, N_k keys, key width d_k, and value width d_v,

$$Q \in \mathbb{R}^{N_q \times d_k}, \quad K \in \mathbb{R}^{N_k \times d_k}, \quad V \in \mathbb{R}^{N_k \times d_v}.$$

Values need not equal keys or have their width. A natural-language question is not the same object as the query vector produced by a learned projection.

Additive attention can use

$$e_{ij} = v_a^\top \tanh(W_q q_i + W_k k_j + b).$$

Scaled dot-product attention uses

$$S = QK^\top/\sqrt{d_k}, \quad A_{ij} = \frac{\exp(S_{ij})}{\sum_m \exp(S_{im})}, \quad C = AV.$$

Scores compare, row-wise softmax normalizes, and value multiplication retrieves. Rows are query destinations and columns are key sources. The context has shape (N_q, d_v).

Under independent zero-mean unit-variance coordinate assumptions, the raw dot-product variance grows with d_k. Scaling by $\sqrt{d_k}$ controls that growth; it is not cosine normalization. Stable softmax subtracts the row maximum before exponentiation without changing normalized probabilities. Before later projections and residuals, the weighted context lies within the convex hull of the available values.

6.3.3 An attention calculation with no hidden steps

Choose teaching vectors

$$Q = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 10 & 0 \\ 0 & 10 \\ 10 & 10 \end{bmatrix}.$$

Raw dot products are $(1, 0, 1)$ and scaled scores are approximately $(0.7071, 0, 0.7071)$. Softmax gives weights $(0.4011, 0.1978, 0.4011)$. The zero raw score does not give zero weight, because $\exp(0) = 1$. Combining corresponding value coordinates gives

$$C_1 = 0.4011(10) + 0.1978(0) + 0.4011(10) \approx 8.0222,$$
$$C_2 = 0.4011(0) + 0.1978(10) + 0.4011(10) \approx 5.9889.$$

Use full precision in code. Change only the query and predict the new ranking. An optional positive temperature divides the scores: below one sharpens weights, above one softens them. The standard example uses one.

6.3.4 Self-attention, cross-attention, and masks

Self-attention projects one sequence X into $Q = XW_Q$, $K = XW_K$, and $V = XW_V$. Cross-attention derives queries from one stream and keys/values from another. An encoder-decoder model uses decoder queries with encoder memory. A decoder-only model normally has no separate source encoder.

In causal next-token training, input position i predicts token $i+1$, so it may access itself and earlier inputs: $j \leq i$. Set future scores to $-\infty$ before softmax, yielding zero weights. Padding masks hide nonexistent tokens instead. Combine permissions and exclude padded query outputs from loss. An all-masked row has no valid normalization; prevent it or specify a deliberate special case. Mask conventions differ across APIs. PyTorch scaled-dot-product attention uses Boolean True for allowed entries, whereas some other attention calls use True for blocked entries.

6.3.5 Benefits, costs, and interpretation

One dense attention layer provides a direct query-to-key computational path to a distant position. Known query operations can run in parallel. Dense score interactions involve T^2 pairs per head: doubling context length quadruples pair count. Exact fused implementations can reduce intermediate memory without changing dense pair-count scaling.

Attention weights describe routing within the computation; they do not prove causal importance. Keep Q and K fixed and change V: weights stay identical but the result can change. Removing and renormalizing a source is a different intervention. Explain what a particular experiment supports rather than interpreting an attractive heatmap as a complete causal explanation.

6.4 Hour 3: assemble and test a Transformer

6.4.1 Tokens and position

Token IDs index learned vectors; their numeric magnitude has no inherent semantic distance. The teaching model has eight symbolic tokens, width 32, four heads, and context length 16. Vocabulary size and embedding width differ. Unmasked attention without positional information is permutation-equivariant: permuting input rows permutes outputs correspondingly. Order must therefore be represented deliberately. Our model adds learned absolute position vectors. The original Transformer used sinusoidal encodings; other position mechanisms also exist. A causal mask already introduces order constraints, so do not apply the unmasked equivariance claim unchanged to a masked model.

6.4.2 Heads, nonlinear transformations, and residual paths

Each head uses learned projections and its own routing weights. Concatenate head outputs and apply a learned output projection. At fixed total width D, H heads have width D/H in this implementation. More heads do not automatically multiply the total width, and heads have no assigned semantic roles. Attention mixes positions, while a shared tokenwise MLP transforms features independently at each position. The notebook expands to $4D$, applies GELU, and projects back to D.

Residual addition requires matching dimensions and preserves an identity route. The teaching decoder uses pre-normalization:

$$u = x + \mathrm{Attention}(\mathrm{LN}(x)),$$
$$y = u + \mathrm{FFN}(\mathrm{LN}(u)).$$

The 2017 paper used a full encoder-decoder with post-normalization and ReLU. Our one-block decoder is an explicitly identified educational variant, not a verbatim copy of that architecture. Its causal self-attention has no encoder-decoder cross-attention sublayer.

6.4.3 Objective, baseline, and generation

Shift targets one token ahead. Logits have shape (B, T, V), and cross-entropy minimizes

$$\mathcal{L} = -\frac{1}{BT} \sum_{b,t} \log p_\theta(y_{b,t} \mid x_{b,\le t}).$$

Pass logits directly to `nn.CrossEntropyLoss`. Training computes many known positions in parallel under the causal mask. During generation, the model predicts one unknown token, appends it, and repeats. Parallel training does not imply arbitrary autoregressive generation is parallel.

Separate synthetic streams use seeds 2026, 2027, and 2028 but share one motif with small random perturbations. This is generalization to new realizations of that process, not transfer to a different engineering domain. A training-only smoothed bigram model is the reference. Uniform prediction over eight tokens has NLL $\log 8 \approx 2.079$ nats/token. Select a checkpoint by validation NLL; evaluate test afterward.

6.4.4 Behavioral checks and limits

Plot an actual trained attention head with query rows and key columns. Upper-triangle zeros are mask constraints, not learned discoveries. Change tokens only at positions 8 onward and compare logits at positions 0–7. They should remain unchanged within the declared tolerance. This tests one concrete causal behavior, not every deployment property. Separately test permutation equivariance of the unmasked operation. Removing positions from an already trained decoder is an inference intervention, not a fair separately trained architecture comparison.

The model generates symbolic states, not evidence-backed natural language. A token generator is only one possible component of an agent. An agent also needs a bounded objective, state handling, validated tools, permissions, stopping rules, and evidence review. Generation alone does not provide factual verification or controlled execution.

6.5 Completion and explain-back

H1 must produce a baseline table, residual plot, a reconstructed learned cell update, and evidence JSON. H2 must produce scores, weights, context, mask assertions, and an interpretation-limiting intervention. H3 must produce a trained-model comparison, head plot, future-token check, and configuration record. Predict the effect of one change before executing, keep the original evidence, and finish with a claim, evidence, limitation, and a test that could challenge the claim.

6.6 Primary sources

- Hochreiter and Schmidhuber (1997), *Long Short-Term Memory*: https://doi.org/10.1162/neco.1997.9.8.1735.
- Bahdanau, Cho, and Bengio (2014/2015), recurrent attention: https://arxiv.org/abs/1409.0473.
- Vaswani et al. (2017), *Attention Is All You Need*: https://arxiv.org/abs/1706.03762.
- Dive into Deep Learning: https://d2l.ai/chapter_recurrent-modern/lstm.html and https://d2l.ai/chapter_attention-mechanisms-and-transformers/transformer.html.
- Executable course notebooks: https://github.com/CHjonnyYu/MLES-Fall-2026.

Chapter 7

Reliability, Error Analysis, and the Midterm Workflow

Weeks 9–11. Reliability connects model results to the populations, operating conditions, inputs and risks that produced them.

Learning outcomes

- Segment error by operating condition and retain sample counts.
- Distinguish a fitted function explanation from a causal claim.
- Preserve exact data, configuration and an executable evidence trail.
- Explain choices and failure cases in writing and orally.

7.1 Week 9: find, explain, change, preserve

Monday, 2 November 2026, 14:00–17:00. Use one complete Exercise notebook throughout the three one-hour modules. H1 has 45 minutes of explanation and 15 of practice; H2 has 15 and 45; H3 has 60 minutes of practice. Eight core tasks take 100 minutes, leaving 20 practice minutes for peer discussion, export and optional work. This is in-class formative practice; designated Homework and written/oral examinations are unchanged.

The goal is to explain where a predictor fails, connect a visible error pattern to a missing relationship, separate two kinds of change and preserve evidence with an honest replay scope. Prerequisites are held-out evaluation, a line and a square, and Python arrays. The core lesson requires no calculus, GPU, paid AI tool, API key or external dataset.

7.1.1 The four questions

1. Which cases are weak, and how many are there?
2. Which missing relationship explains the diagnostic pattern in this constructed example?
3. Did the input cases change, or did the target relationship change?

4. Which parts of the saved result can another person actually check?

A successful execution is evidence about this code and these observations. It does not establish validity for a physical system or deployment. Although these checks also matter for LSTM and Transformer predictions, we use transparent synthetic regression to make the mechanism visible.

7.1.2 Use the notebook independently

Save a copy, run Setup once and complete Tasks 1–8 in order. Each task provides Markdown instructions, a prediction cell, executable scaffold, stopping condition and explanation cell. The worked notebook is an alternative with example responses, not another prerequisite. If the runtime resets, run the same file from the top. Save the edited notebook and download the ZIP before the temporary runtime ends.

7.2 Task 1: five errors before a model library

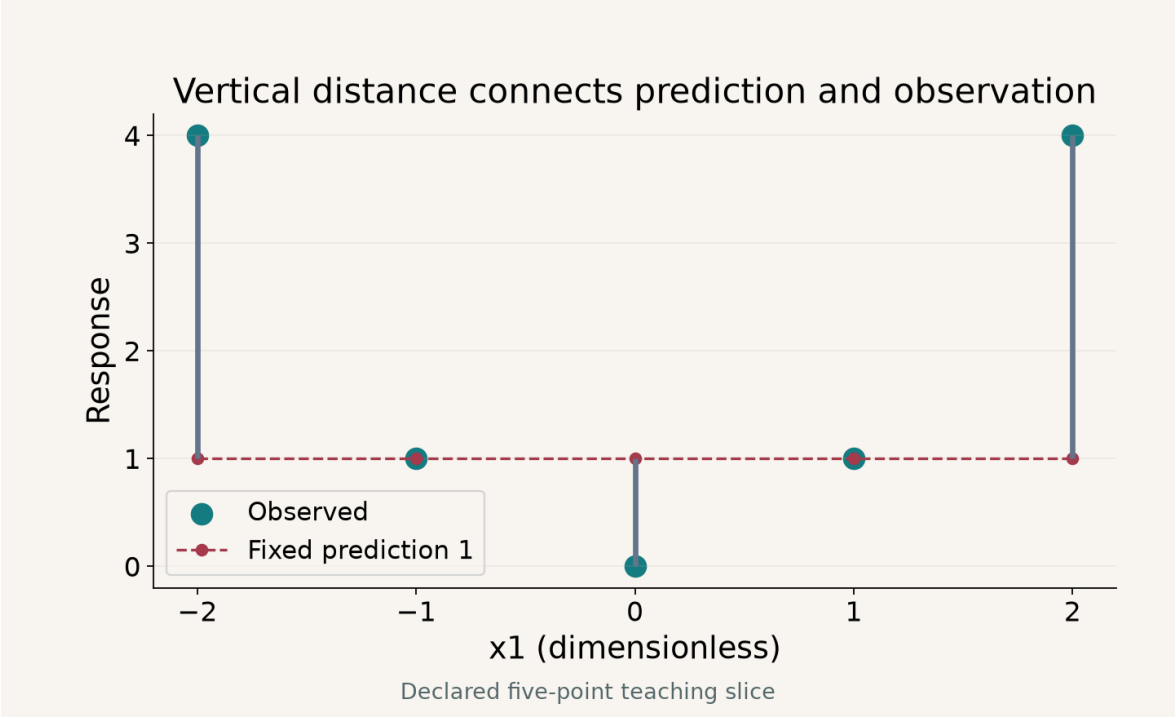
We declare the residual sign $e_i = \widehat{y}_i - y_i$: positive means too high and negative means too low. Hold $x_0 = 0$ and noise at zero. The known synthetic response is x_1^2. A deliberately fixed horizontal prediction of 1 gives:

x_1	Observation	Prediction	Residual	Absolute error
-2	4	1	-3	3
-1	1	1	0	0
0	0	1	1	1
1	1	1	0	0
2	4	1	-3	3

This horizontal prediction was not fitted to these five points and is not claimed optimal for them. It is an explicit teaching approximation, separate from the pipelines trained in Task 3.

$$\sum_i |e_i| = 7, \quad \sum_i e_i^2 = 19, \quad \text{MAE} = 7/5 = 1.4, \quad \text{RMSE} = \sqrt{19/5} \approx 1.949.$$

Both MAE and RMSE have response units; MSE has squared response units. Squaring makes a large error contribute disproportionately before the final square root. Neither metric is universally best: state the task and error costs. For the same equally weighted rows, RMSE is at least MAE.



Noise-free five-point teaching slice. Vertical segments connect the declared prediction with the observed square.

Predict and explain. Which rows are underpredicted? At $x_1 = -2$, explain why $1 - 4 = -3$ is a low prediction. Show the intermediate sums rather than calling a metric function only.

7.3 Task 2: the mixture can change the score

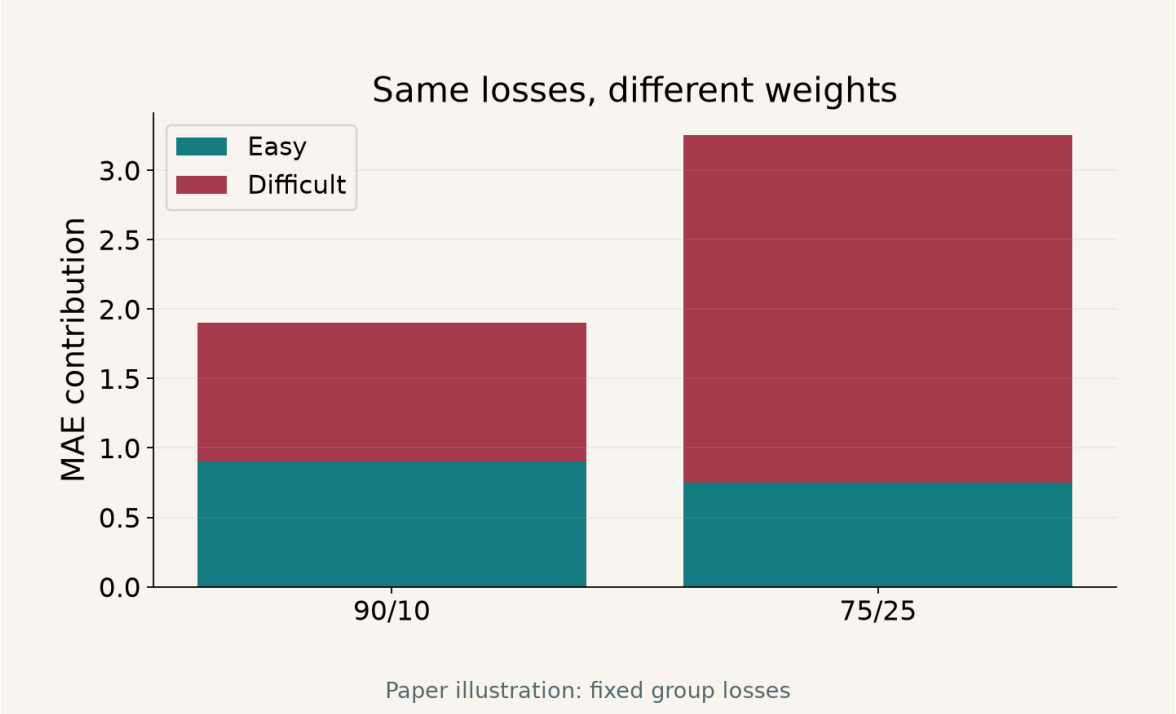
This paper illustration is separate from the 75-row test set used later. Suppose 90 easy cases each have absolute error 1 and 10 difficult cases each have error 10. Then

$$\text{MAE} = \frac{90 \times 1 + 10 \times 10}{100} = 0.9 + 1.0 = 1.9.$$

Change only the counts to 75 easy and 25 difficult cases, keeping 100 total and both group losses fixed:

$$\text{MAE} = \frac{75 \times 1 + 25 \times 10}{100} = 0.75 + 2.5 = 3.25.$$

The changed aggregate does not require changed within-group errors. More generally, for group fraction q and these two fixed losses, $\text{MAE}(q) = (1 - q)1 + q10 = 1 + 9q$. In real observations, a changed aggregate alone does not identify which mechanism changed.



Stacked contributions separate each group loss from its prevalence. The interactive slide changes the difficult-case count only.

Stopping condition. Explain which two quantities stayed fixed and which changed. Do not report 90/10 or 75/25 as the actual fitted-model test counts.

7.4 Task 3: one transparent main experiment

The notebook generates $n = 500$ independent synthetic rows. Each row contains dimensionless inputs x_0, x_1, x_2 drawn independently from a standard normal distribution. Noise is independent with standard deviation 0.2:

$$y = 2x_0 + x_1^2 + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, 0.2^2).$$

x_0 contributes linearly, x_1 contributes through its square and x_2 is a negative control unused by the generator. The response has synthetic response units. These are neither measured equipment data nor financial forecasts. Knowing the constructed generator permits a representation explanation without claiming physical discovery.

Before fitting, seed 2026 fixes the generated sample and first split. A 70/30 split yields 350 training IDs and 150 held IDs; seed 2027 divides held IDs equally into 75 validation and 75 test IDs. Sort IDs within splits, assert disjointness and save exact CSV bytes with row ID and membership.

7.4.1 Declare the candidates before inspecting test outcomes

1. Linear: raw three inputs, training-fitted standardization, Ridge with $\alpha = 1$ and SVD solver.
2. Quadratic: append the deterministic feature x_1^2, then the same training-fitted standardization and Ridge configuration.

Centering/scaling is affine and cannot create a missing square. Adding the square is a change of feature basis. Both scalers and regression coefficients fit only the 350 training rows. Choose the candidate with smallest validation MAE; exact ties favor linear. Save that choice before displaying test metrics. The predeclared two-model test comparison diagnoses errors but does not reopen selection.

7.4.2 A measured reference, not a score to copy

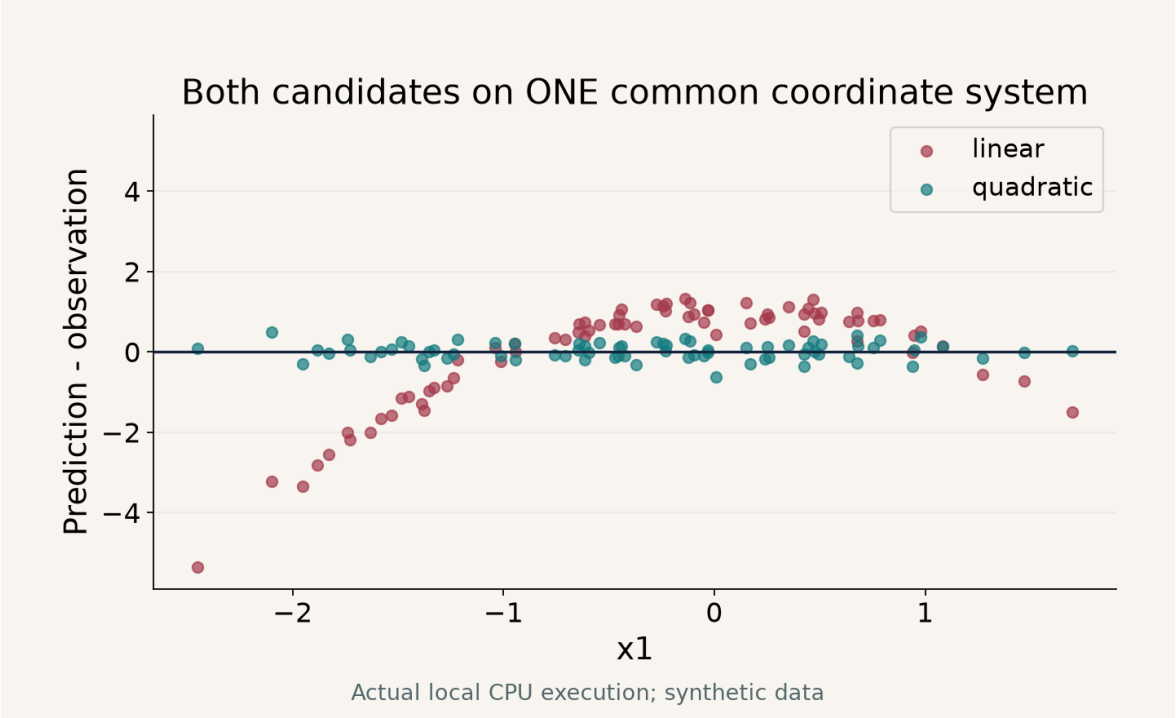
The fresh local CPU run on 27 October 2026 produced the following rounded values. Package versions are exported; tiny differences across environments are possible.

Candidate	Validation MAE	Test MAE	Test RMSE
linear	1.017086	1.031530	1.319474
quadratic	0.158188	0.164319	0.205100

The quadratic candidate was selected on validation. Do not copy these values into an unexecuted response. State the split, loss, data identity and selection rule alongside any claim.

7.5 Task 4: make a fair visual comparison

Plot prediction minus observation against x_1 for both candidates on the exact same 75 test rows. Use a shared horizontal range and a shared vertical range. Compare magnitude before opening the separately labeled quadratic zoom. Automatic axis scaling can make unequal errors fill two panels equally; a zoom answers a different question.



Both candidates occupy a single shared coordinate system. The linear residual leaves a visible curved pattern; the square-feature candidate has much smaller residuals.

The residual curve suggests a missing relationship. Here the square is known by construction. A curved observational residual alone does not prove physical causation.

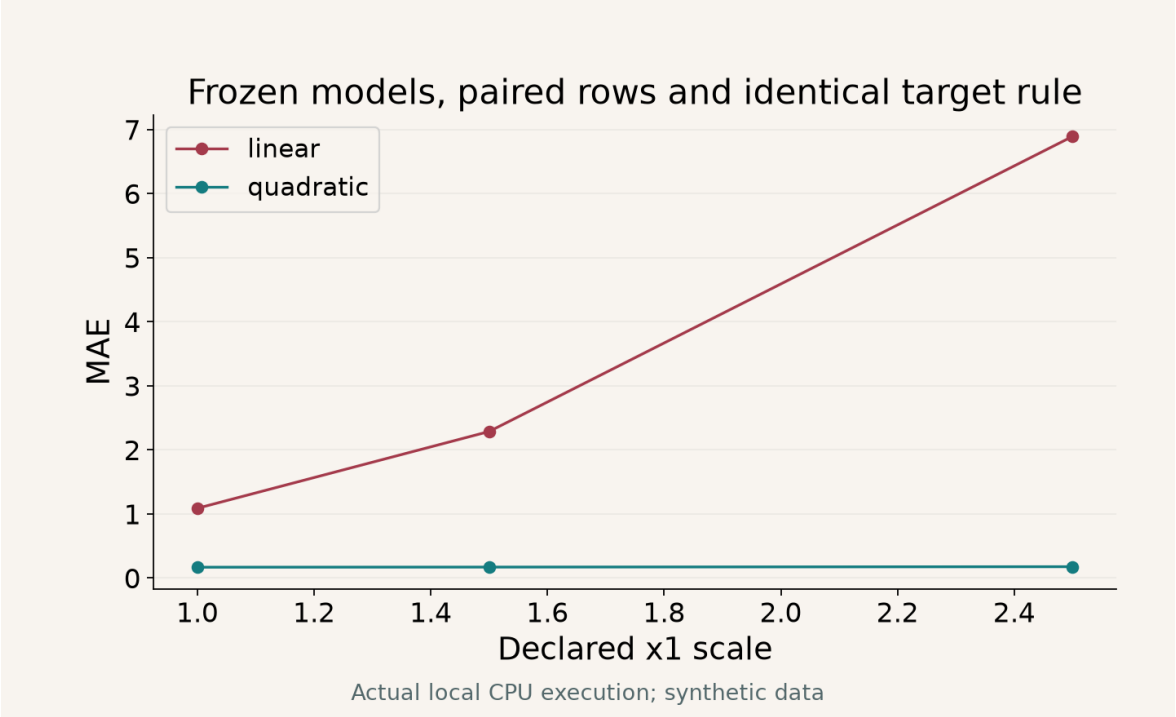
Keep the predeclared groups $|x_1| \leq 1$ and $|x_1| > 1$ fixed. Diagnose the linear candidate first, then the quadratic candidate on the same slices. Each candidate has 50 inside and 25 outside test rows. In this run, inside/outside MAE is approximately 0.775/1.545 for linear and 0.174/0.144 for quadratic. The quadratic outside group is not worse in this sample: do not force a universal claim about extreme cases.

Stopping condition. Show actual counts and group losses for both models; counts sum to 75. Do not optimize a new boundary on final test labels. Exploratory findings need a new confirmation set before renewed performance claims.

7.6 Task 5: input change with an unchanged rule

Use a separate seed 2126 to generate 150 paired base rows and one noise realization. Predeclare x_1 scale factors 1, 1.5 and 2.5. At each scale, change only x_1, hold x_0, x_2, row pairing and noise fixed, and recompute outcomes with the original rule $y = 2x_0 + x_1^2 + \epsilon$. Predict with both frozen candidates without refitting.

Why recompute labels? The original relationship must remain fixed. Changing a feature encoding while keeping old labels would test a different problem. Why pair rows? It makes the controlled factor explicit; it does not establish physical realism.



Actual labeled MAE at the three declared input scales. The two candidates are frozen; no training occurs during the sweep.

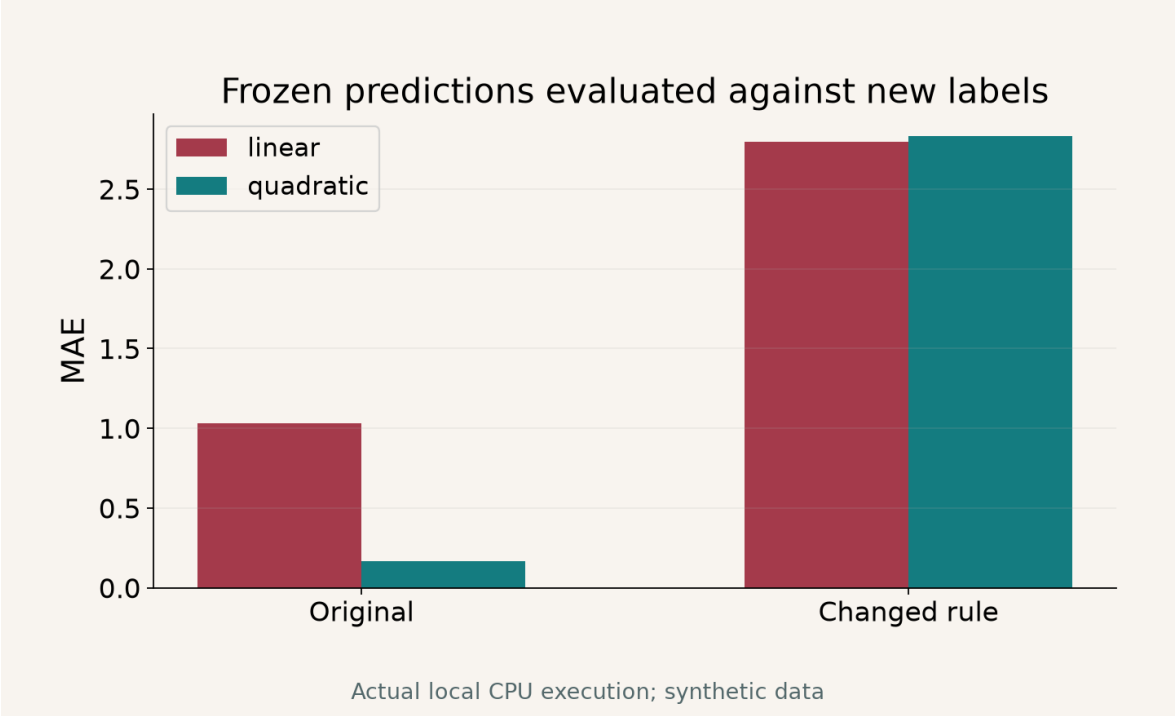
An input histogram establishes a changed input distribution in this construction; it is not an accuracy metric. Accuracy needs outcomes and a declared loss. The effect of input shift depends on the task and fitted function, so do not claim all shifts must harm all models. Snapshot recorded scaler and regression parameters before the sweep and assert exact equality afterward. This establishes unchanged recorded parameters, not arbitrary implementation state.

7.7 Task 6: unchanged inputs, changed target relationship

Keep the exact test inputs, row order, noise realization and existing predictions. Replace only the target rule:

$$y_{\text{new}} = -2x_0 + x_1^2 + \epsilon.$$

The input bytes and input histogram are identical by construction. The predictor also stays fixed. Only the outcomes against which it is evaluated change. Hash the input array before/after, assert identical predictions and recompute labeled errors.



The original and changed-rule errors use the same inputs and frozen predictions. Only target outcomes differ.

This constructed conditional change cannot be detected by monitoring inputs alone. Outcomes reveal its effect. It does not prove every possible relationship change must worsen every metric. In real systems, outcomes may arrive late or selectively; recording input drift and recording labeled performance are separate obligations.

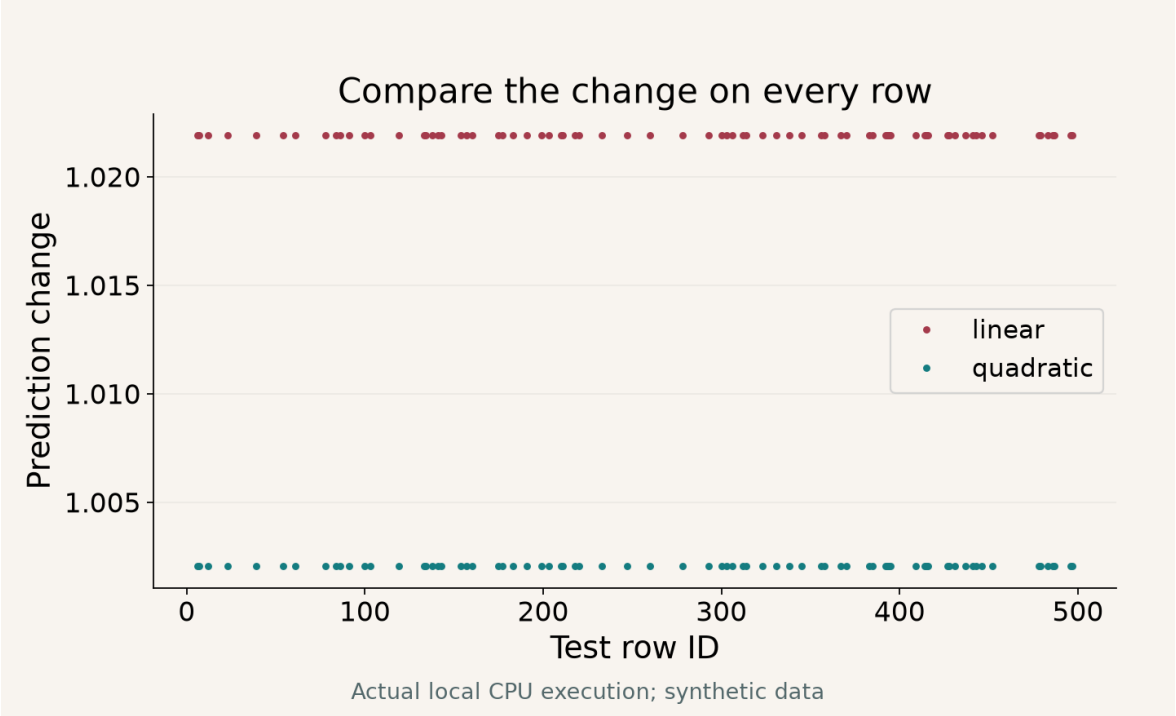
Explain the distinction. Task 5 changes cases while preserving the rule; Task 6 preserves cases while changing the rule. Both keep model parameters fixed. Identify the matching evidence rather than naming both situations only as "drift."

7.8 Task 7: check an explanation of the fitted function

Increase original-coordinate x_0 by 0.5 while holding all other inputs fixed. Standardization uses $z_j = (x_j - \mu_j)/s_j$. If the standardized Ridge coefficient is θ_0, the raw-coordinate slope is $\beta_0 = \theta_0/s_0$. In these additive models, no square or interaction contains x_0, so every row has

$$f(x + 0.5e_0) - f(x) = 0.5\beta_0.$$

Predict this difference before calling the model. Compare the entire 75-row difference vector against the formula, not just its mean. The numerical check uses $|a - b| \leq 10^{-10} + 10^{-10}|b|$.



Observed prediction changes for every test row when only x0 increases by 0.5. Each model has its own fitted raw-coordinate slope.

No new target labels enter this check. It verifies a property of the fitted mathematical function and the coordinate convention. It is neither a physical intervention nor a new accuracy estimate. A standardized coefficient refers to one standardized feature step; do not apply it directly to a raw-input step.

7.9 Task 8: CSV readback and an honest handover

Read the exact frozen CSV bytes and compare their SHA256 with the saved reference. Parse and sort by stable row ID; assert 500 unique IDs and the original split membership. Clone the already-selected candidate, fit only CSV training rows and compare the full, aligned 75-row test prediction vector with the reference. This run produced maximum absolute difference zero, under the declared tolerance.

Scope: same-kernel CSV readback consistency. Model definitions, configuration, selected candidate and installed dependencies still come from the current kernel. This is not independent-process or fresh-environment reproduction. Week 10 extends the execution contract. A checksum means matching bytes against a reference; it does not authenticate the sender or establish scientific correctness.

7.9.1 Export and explain-back

Complete 19 response fields: eight predictions, eight explanations, controlled change, limitation and AI-use disclosure. Empty fields produce a draft. Nonempty fields mean ready for peer discussion, not automatically correct. The ZIP contains frozen CSV, row-aligned predictions, metrics, configuration, versions, figures, responses, supplied source snapshot and hash manifest. Save the edited notebook separately if you changed code.

A partner should select one actual result and ask: what changed, what stayed fixed, what supports your explanation and what remains untested? Locate the matching exported record. A ZIP that opens successfully does not itself establish a correct model or conclusion.

7.9.2 Troubleshooting

A missing variable usually means a skipped cell or runtime reset: run from Setup in numeric order. Keep assertion failures rather than copying a reference metric. Compare data identity, row alignment, configuration and package versions before attributing a difference to the algorithm. Download the ZIP through the Colab Files sidebar or Jupyter file link before leaving the temporary runtime.

7.9.3 Summary

Find weak cases before trusting an average. Explain the missing shape using a checked mechanism. Separate input changes from target-rule changes. Test an explanation against the fitted function. Preserve evidence and name the replay boundary.

7.10 Supplementary reasoning: optional, not extra core tasks

7.10.1 Conditional risk and weighted groups

For a fixed fitted function f, declared loss L and joint distribution P, population risk is $R_P(f) = \mathbb{E}_P[L(Y, f(X))]$. A finite test mean estimates this quantity only under sampling assumptions. A result on P does not automatically transfer to Q. For exhaustive groups, population risk is a prevalence-weighted sum of conditional risks; finite group MAE uses the same identity exactly. Group RMSE must be pooled through squared errors:

$$\mathrm{RMSE}_{\mathrm{all}} = \sqrt{\frac{\sum_g n_g \mathrm{RMSE}_g^2}{\sum_g n_g}},$$

not a simple mean of group RMSE values.

7.10.2 Why the population linear approximation misses the square

For independent standard-normal coordinates and mean-zero noise, the least-squares projection of $2x_0 + x_1^2 + \epsilon$ onto an intercept and linear inputs is $1 + 2x_0$: $\mathbb{E}[x_1^2] = 1$, $\mathbb{E}[x_1^3] = 0$ and independent cross-covariances vanish. Its residual is $1 - x_1^2 - \epsilon$. This explains the curved pattern. The finite, regularized Ridge fit is not exactly this population projection; measured coefficients and residuals need not equal the ideal formula. The Task 1 fixed line is a noise-free visual slice of this motivation, not a fit to its five points.

Normal distributions have unbounded support. Scaling x_1 changes its distribution and the frequency of tail cases, not its theoretical support from finite to infinite. A finite training sample still leaves sparsely observed regions. Describe coverage honestly rather than claiming every wider case is outside normal support.

7.10.3 Recovering original-coordinate coefficients

For feature vector $\phi(x)$, standardized predictor $b_s + \sum_j \theta_j(\phi_j - \mu_j)/s_j$ has raw coefficients $\beta_j = \theta_j/s_j$ and intercept $b = b_s - \sum_j \theta_j \mu_j / s_j$. This coordinate conversion explains Task 7; it does not turn association into causality.

7.10.4 Optional A: encoding check

Multiply the dimensionless x_0 encoding by 100 while retaining original targets and frozen models; measure the error, then decode by dividing by 100 and compare aligned predictions. This tests a representation mismatch. It is not a physical unit conversion and differs from Task 5, which deliberately recomputes outcomes to preserve the target rule.

7.10.5 Optional B: fixed-model bootstrap

Within each fixed slice, sample absolute errors with replacement 300 times and report percentile bounds for their mean. This assumes IID cases within the slice and conditions on the already

fitted function. It excludes training-set variation, arbitrary future distribution change and many measurement errors. A narrow interval does not remove those limits. Optional results remain separate from the eight-task completion count.

7.11 Primary references

- Monitoring ML pipelines: https://developers.google.com/machine-learning/crash-course/production-ml-systems/monitoring.
- Leakage-safe pipelines: https://scikit-learn.org/stable/common_pitfalls.html.
- Regression metrics: https://scikit-learn.org/stable/modules/model_evaluation.html#regression-metrics.

References checked 27 October 2026. All examples and data are synthetic teaching fixtures.

7.12 Week 10: Reproducible ML Workflow Studio

Monday, 9 November 2026, 14:00–17:00. The class contains 60 minutes of explanation and 120 minutes of practice. H1 uses 45/15 minutes, H2 uses 15/45, and H3 is a 60-minute studio. Eight core exercises and two optional challenges are formative in-class practice, not new weekly graded homework. The next scheduled meeting is the written and oral midterm examination; this studio does not replace either examination.

The goal is a portable handover: another interpreter receives frozen files, checks their contract, reconstructs a declared model, and compares predictions by stable row identity. A matching screenshot, aggregate score or second notebook variable is not sufficient. Week 9 studied where a model fails; Week 10 asks whether the evidence supporting that analysis can travel.

7.12.1 Computational replay has a bounded meaning

Here, replay means recomputing predictions from the same frozen input records and declared procedure. A new interpreter and working directory remove the notebook's global variables and fitted-object memory. They do not create a new dataset, machine, package environment or physical experiment. Thus numerical agreement is narrower than independent scientific replication. A reproducible model can still be inaccurate or inapplicable.

A two-row counterexample clarifies the distinction between score and prediction identity. Errors $[0, 2]$ and $[2, 0]$ both have MAE 1, but fail on different observations. Compare aligned prediction vectors, not only their mean loss.

7.13 Stable identity, not a seed alone

A seeded permutation selects positions in an ordered array. If the array order changes first, the same positions may identify different observations. Assign source IDs before shuffling and persist them

with their measured values. To reconstruct a cohort, build an ID-keyed lookup and iterate over the saved ID sequence. A set comparison verifies membership; a sequence comparison additionally verifies alignment.

The partition contract requires unique row IDs, no duplicates within any split, pairwise disjoint train/validation/test sets, the expected counts, and a union equal to the full frozen dataset. Disjointness alone does not catch omitted rows. Source row IDs are version-specific dataset identifiers, not globally stable patient identities or predictive features.

7.14 The packaged regression fixture and its limits

The original Week 10 material used scikit-learn's bundled diabetes regression dataset. We retain its 442 observations and 10 numeric inputs but call `load_diabetes(scaled=False)`. The default loader returns globally centered/scaled inputs; raw loading makes the boundary of training-only fitted preprocessing explicit. We do not assign medical interpretations to individual features.

Use boundary: this is a statistical workflow exercise, not diagnosis, treatment guidance, causal discovery or external clinical validation. A random-row teaching split cannot establish future-time, new-site or clinical deployment validity. Dataset documentation also cautions that some feature meanings are not fully explicit.

One seeded permutation reserves 265 training, 88 validation and 89 final-test rows. Store exact floats, feature names, row IDs and split lists. The notebook checks finite numeric values, shape, counts and ID membership before fitting.

7.14.1 An executable interface

The CSV header must exactly match `row_id`, the ten declared feature names in canonical order, and `target`. Duplicate IDs, non-finite values and reordered columns are rejected. Strict rejection is appropriate for a positional model interface; a separately declared and tested name adapter may create a valid canonical interface. Silently guessing feature order is not an acceptable repair.

These checks establish well-formedness relative to a contract, not scientific validity. A finite label can still be wrong, a unique ID can be attached to a wrong observation, and a correctly shaped dataset can be biased.

7.15 Baseline, selection and held-out evidence

Declare four candidates before final evaluation: a training-target median baseline and training-scaled Ridge models with $\alpha \in \{1, 20, 100\}$. A median minimizes $\sum_i |y_i - c|$ over constant predictions c. For an even sample size, any value between the two middle order statistics is a minimizer. That training property does not guarantee the best held-out score.

For the standardized design matrix Z, scikit-learn Ridge uses a residual-sum-of-squares objective plus an L2 penalty:

$$\min_{b,\beta} \sum_{i \in \text{train}} (y_i - b - z_i^\mathsf{T}\beta)^2 + \alpha \sum_j \beta_j^2.$$

The intercept is not penalized in this formulation. Standardization and all coefficients are fitted on training data only. Stronger regularization changes the fitted function and coefficient norm; validation MAE need not improve monotonically. The training objective and selection metric need not be identical.

Choose the lowest validation MAE, breaking exact ties by the declared order: median, Ridge 1, Ridge 20, Ridge 100. Freeze the selected candidate before opening final-test results. The replay program refits this already selected candidate on training rows; it does not redo selection or refit on training plus validation.

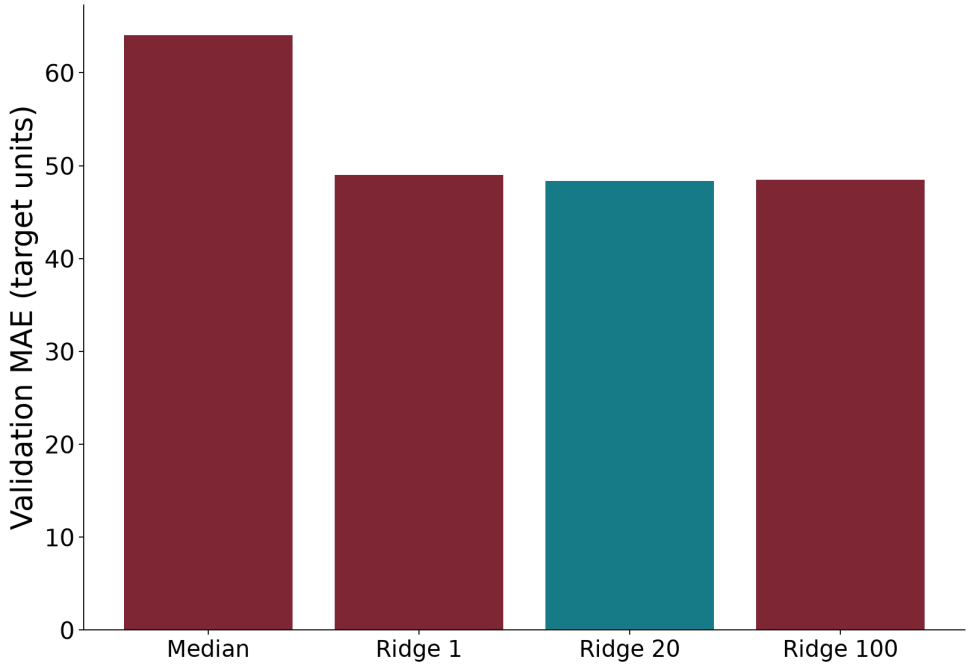


Figure 7.1: Actual validation comparison on 88 rows. The reference run selects Ridge 20. Students retain their own actual environment and outputs.

Report the selected candidate and the predeclared median baseline on the same 89 final-test observations. The residual convention is prediction minus observation; positive means overprediction. Inspect one actual large-residual source ID and its observed/predicted values. Do not infer a clinical mechanism from that row or tune alpha after inspecting it.

Candidate	Test n	MAE	RMSE
median	89	65.280899	79.418957
ridge-20	89	48.325756	59.411283

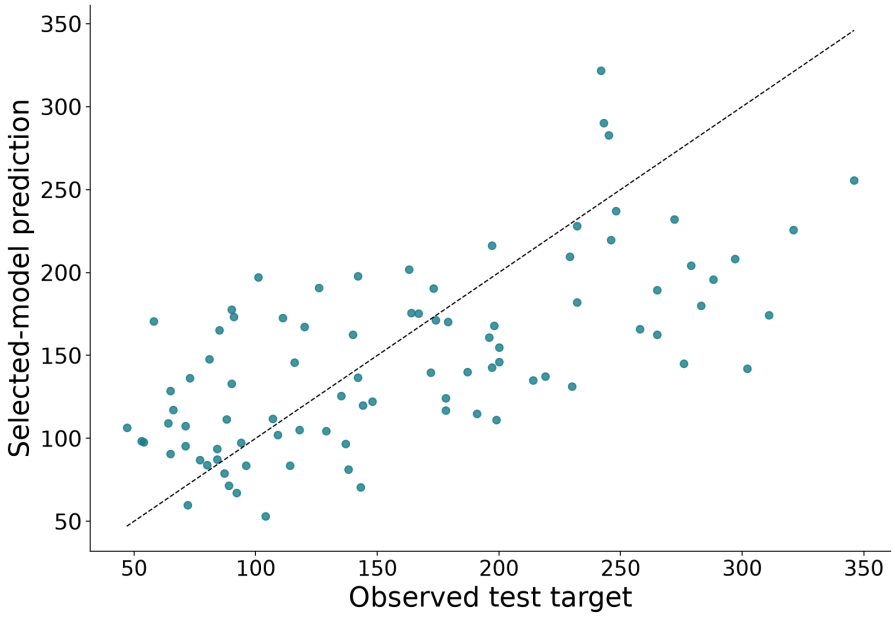


Figure 7.2: Actual selected-model final-test predictions. A later perfect computational replay does not remove these prediction errors against outcomes.

7.16 A checkable function explanation

For $z_j = (x_j - \mu_j)/s_j$, standardized coefficient β_j gives

$$\beta_j^{\text{raw}} = \frac{\beta_j}{s_j}, \qquad b^{\text{raw}} = b - \sum_j \frac{\beta_j \mu_j}{s_j}, \qquad f(x) = b^{\text{raw}} + \sum_j \beta_j^{\text{raw}} x_j.$$

Reconstruct predictions and compare them to the pipeline. Increasing one input by $0.1 s_j$ changes the fitted prediction by $0.1\beta_j$, with other coordinates fixed. This is an exact property of the additive fitted function, up to numerical tolerance. It need not correspond to a feasible causal or clinical intervention, especially with correlated features.

If the selected candidate were the constant median, this feature interpretation would not apply. The notebook then inspects the separately predeclared Ridge-20 candidate and records its name. Do not silently interpret a different candidate as the selected one.

7.17 What belongs in the handover

The payload has six required files:

File	Role
`frozen.csv`	Exact source IDs, features and target
`split.json`	Ordered training, validation and test IDs
`config.json`	Feature order, choices, selected candidate, versions
`replay.py`	Readable validation and fitting procedure
`reference-predictions.csv`	ID-aligned reference outcomes/predictions
`result.json`	Validation/test evidence and limitations

A separate `manifest.json` records the SHA-256 of each payload. It does not hash itself, which would be circular. The exported ZIP also retains actual plots and the fresh-process result.

7.17.1 Integrity is not authentication

SHA-256 detects changed bytes relative to a known reference digest. Parsing records checks a different identity. Appending a blank CSV line can preserve parsed rows while changing its hash. An unsigned manifest does not authenticate an unknown sender: someone able to replace both payload and digest can create another internally consistent package. Neither hashes nor schemas validate scientific labels or a performance claim.

The lab avoids loading a model pickle by reconstructing from CSV, JSON and readable source. Pickle-based loaders may execute arbitrary code. Executing Python source is also consequential: review and trust the generated runner before running it. A checksum does not make arbitrary downloaded code safe.

7.18 Replay outside notebook memory

The notebook launches a new Python interpreter in a new temporary working directory, passing only an explicit handover path and output path. The runner first verifies the required-file set and hashes, then validates schema and split membership, then refits the frozen candidate and predicts the saved test IDs. Notebook globals cannot rescue missing files.

Check exact test-ID sequence before numerical comparison. For each value, the comparison uses

$$|p_i^{\text{replay}} - p_i^{\text{reference}}| \le a + r|p_i^{\text{reference}}|, \qquad a = 10^{-8},\ r = 10^{-10}.$$

For reference value 10, the allowed difference is 1.1×10^{-8}. This tolerance is a numerical-agreement rule, not a model-accuracy score. Inspect input bytes, IDs, feature order, configuration and versions before changing it.

The actual local reference maximum prediction difference is 0.0. That establishes a replay under the recorded installed environment, not all future software versions. NumPy's random-stream compatibility documentation imposes strict conditions involving seed, generator, call sequence, build, environment and machine; do not overstate a seed-only guarantee.

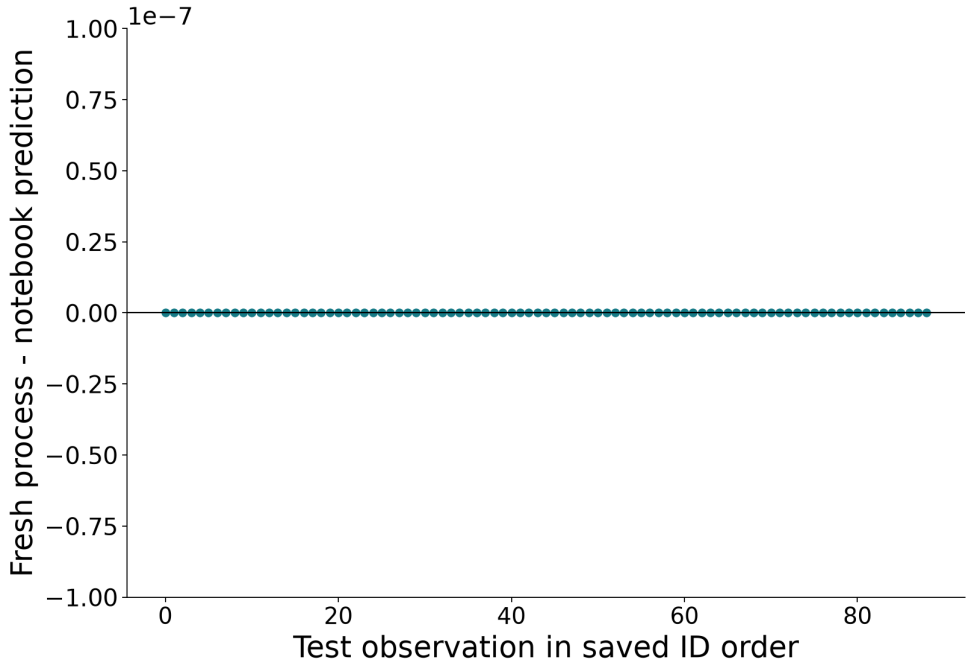


Figure 7.3: Actual fresh-process prediction differences after exact ID alignment. The vertical scale is explicit. Replay difference and residual against observed target answer different questions.

7.19 Test deliberate failures on copies

Corrupt only temporary copies of the data. Inject a duplicate ID, a non-finite value, or a reordered header. Catch the expected `ValueError` and record its message. An accepted malformed copy must trigger a failed assertion. Recheck the original data hash afterward.

7.20 Eight core exercises and two optional challenges

Each task requires a prediction before running, an actual observation, an explanation and a stopping check. Alternate driver and reviewer in pairs. Use H1 for Tasks 1–2, guided H2 for Tasks 3–5, and H3 for remaining work, review and export. These tasks share the 120 minutes of practice; they are not extra hours after the walkthrough.

1. **Row-order trap:** reverse a 20-ID toy dataset, reuse seeded positional indices, then recover frozen membership by ID. Explain set equality versus sequence equality.
2. **Preprocessing scope:** assert the fitted scaler mean equals training means. Contrast all-row means descriptively without fitting a leaky competing model or claiming a particular score change.
3. **Selection rule:** recompute the validation-MAE winner and declared tie order. Never use final-test results to choose it.
4. **Contract failures:** reject duplicate IDs, NaN and reordered columns on temporary copies. Confirm the original CSV unchanged.

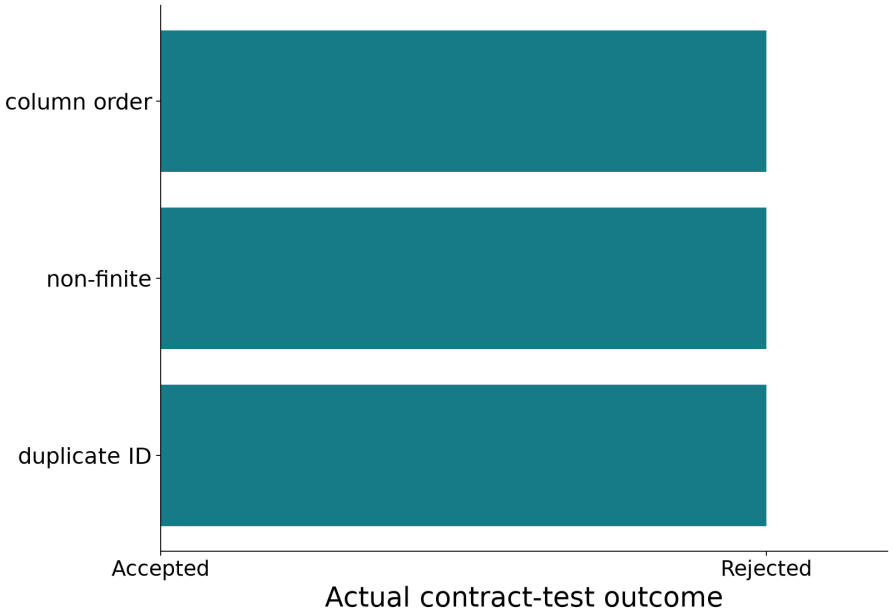


Figure 7.4: Actual three-case contract rejection test. These outcomes test an interface, not clinical safety or predictive accuracy.

5. **Fresh process:** execute the reviewed runner in another temporary working directory. Compare saved ID sequences and predictions under the explicit tolerance.
6. **Controlled extension:** fit alpha 200 on the same training rows and report validation only. Because final-test results have been seen, label this exploratory; a new selection claim needs a fresh evaluation protocol. Preserve the original winner.
7. **Local explanation:** add 0.1 training SD to one declared feature and verify the fitted-function prediction change. Record the candidate and exclude causal or clinical claims.
8. **Handover audit:** verify every payload digest and identify one actual failure row. Write claim, evidence, limitation and next evaluation so a partner can locate the supporting files.

Optional A changes a copied CSV and confirms manifest rejection before training. Optional B explicitly restores reordered numeric columns by unique names and verifies exact recovery of the canonical matrix. Both preserve the reference package and neither is required for core completion.

7.20.1 Student responses and evidence export

The 19 core fields comprise eight predictions, eight explanations, controlled change, limitation and AI-use/checking disclosure (or "none"). Published responses and outputs start blank. Missing fields produce status `draft`; nonempty fields mean `ready for peer discussion`, not a correctness grade. Rerun only the final export after editing responses; initialization resets them.

Download both the handover ZIP and exercise ZIP before the free runtime ends. After code review, the extracted runner can be executed in a compatible environment using the explicit root and output paths shown in the notebook. No paid AI, GPU, API key or external dataset download

is needed. AI may help explain code or suggest hypotheses, but it must not fabricate execution evidence or measured values.

7.21 Instructor execution and limits

On 19 October 2026 both notebooks were fully executed in fresh local CPU Jupyter kernels. Each invoked independent replay interpreters. All eight core tasks, both optional challenges, draft/completed export branches, 11 plot exports, numerical checks and ZIP integrity were checked. This is not a claim that the Google Colab browser itself was used for execution. Actual versions are recorded in the evidence JSON; a local software check does not establish clinical, causal or external deployment validity.

Reference CSV SHA-256:

`a589d3035403d59869ec18c1007e379a396c9cc53b562cb979be676f2b85c4d4`

7.22 Summary and examination bridge

Freeze identities, choices, transformations and environment. Reject malformed interfaces before fitting. Compare aligned prediction vectors in a new process, then state exactly what agreement supports. Keep predictive error, replay error, file integrity and scientific validity separate. For the written and oral midterm, be ready to explain one choice, one failure and one change that would invalidate the claim.

7.23 Primary-source reading

scikit-learn: raw dataset API

scikit-learn: preprocessing pitfalls

scikit-learn: persistence and security

NumPy: random-stream compatibility

7.24 Weeks 10–11: workflow and examination preparation

The workflow studio assembles a problem brief, data contract, train/validation/test rule, baseline, metric, error analysis and bounded recommendation. Existing designated Homework requirements remain separate from formative weekly exercises. The written examination checks conceptual understanding. The oral examination asks students to explain choices, interpret a failure case and identify one change that would invalidate a result. Practice notebooks do not replace the written or oral examination.

Checkpoint

A model has a strong overall score but fails near a sensor detection limit. State one evaluation slice, one data-collection action and one possible operating restriction. Name the additional evidence needed before recommending that restriction.

Part II

Language Models, Agents, and Engineering Workflows

Chapter 8

LLMs, bounded agents, and vibe coding

Week 12: 2026-11-23, Monday 14:00–17:00.

Train a transparent character bigram language model, then inspect a separate deterministic proposal validator. Neither the bigram nor the rule dispatcher is a Transformer or a live LLM agent.

Learning outcomes

Explain next-token prediction; validate structured tool proposals; retain control outside the model.

The meeting contains three 60-minute modules: 60 minutes explanation and 120 minutes practice. The eight core exercises and two optional challenges are in-class practice, not additional weekly graded homework.

8.1 H1: From next-token prediction to tool proposals

8.1.1 Language is an interface, not an oracle

Generate a proposal, then verify the claim outside the model.

Worked reasoning. Token probabilities; sampling; structured proposals; validation; Task 1 prediction.

Check. What could make fluent output incorrect?

Explanation and boundary. A likely token sequence can lack factual evidence, correct arithmetic, or permission. Separate these checks.

8.1.2 A token is not necessarily a word

A tokenizer maps text to integer IDs using a specified vocabulary.

Worked reasoning. Character demo: hot = h,o,t; a production subword tokenizer can split differently. Preserve tokenizer identity.

Check. Can one word always be counted as one token?

Explanation and boundary. No. Token boundaries depend on the tokenizer and text. The notebook uses characters, not a production LLM tokenizer.

8.1.3 Autoregression factorizes a sequence

Each conditional distribution uses the preceding context only.

Worked reasoning. p(A)=0.5; p(B given A)=0.6; p(C given AB)=0.2; joint probability=0.06.

Check. Does the factorization prove the sequence is true?

Explanation and boundary. No. It describes sequence probability under a model, not truth. The multiplication follows the chain rule.

8.1.4 Cross-entropy penalizes surprise

Training lowers negative log probability on observed next tokens.

Worked reasoning. If p=0.8 then -ln(p)=0.223; if p=0.2 then loss=1.609. Mean=0.916 nats.

Check. Why is assigning low probability to the observed token costly?

Explanation and boundary. Negative log grows as probability approaches zero. Mean teacher-forced loss is not factual verification.

8.1.5 Attention selects context-dependent mixtures

Causal masking prevents a position from reading later tokens.

Worked reasoning. At position 2, keys 1 and 2 are allowed; keys 3 and 4 are blocked before softmax.

Check. Does attention provide an external source of facts?

Explanation and boundary. No. It mixes representations from supplied context. Retrieved evidence and its quality remain separate responsibilities.

8.1.6 Pretraining and post-training differ

Pretraining learns broad prediction; post-training changes response behavior.

Worked reasoning. A polite response with well-formed JSON can contain an incorrect number or an unsupported citation.

Check. Does instruction tuning remove the need for tests?

Explanation and boundary. No. Behavior adaptation is not a guarantee of accuracy, authorization, or grounding.

8.1.7 Sampling changes a distribution

Temperature divides logits before softmax; it is not a truth control.

Worked reasoning. Logits [2,0]: T=1 gives [0.881,0.119]; T=2 gives [0.731,0.269].

Check. What does increasing temperature change here?

Explanation and boundary. It flattens this distribution and raises the lower-logit token probability. It does not make the content more or less factual by definition.

8.1.8 A tiny language model is inspectable

Character bigrams estimate the next character from one previous character.

Worked reasoning. Run the fixed original teaching corpus. Find the most frequent successor of a space and compare its count with its probability.

Check. Is this a miniature Transformer?

Explanation and boundary. No. It is a trained conditional-count language model with one-character context; it lacks attention and contextual embeddings.

8.1.9 Prediction before sampling creates evidence

Keep corpus, vocabulary, smoothing and random seed with the generated text.

Worked reasoning. Predict whether lower temperature concentrates probabilities; compare entropy and sampled continuations without changing the corpus.

Check. Must a lower-entropy model produce a better answer?

Explanation and boundary. No. Entropy measures distribution concentration. Quality needs a task-specific evaluation independent of sampling sharpness.

8.1.10 Summary: separate probability and proof

A language model proposes tokens; verified tools establish bounded numeric results.

Worked reasoning. Show one next-character distribution, one computed loss and one limitation of the bigram model.

Check. Which component will authorize the next tool call?

Explanation and boundary. Deterministic validation outside the model. A model probability cannot grant permission.

8.2 H2: Structured outputs and bounded tool execution

8.2.1 The tool boundary is executable code

A proposal becomes an action only after deterministic validation.

Worked reasoning. Tasks 3-5: valid JSON, invalid JSON, tool allowlist, numeric checks and a finite attempt budget.

Check. Why not execute a plausible tool name directly?

Explanation and boundary. A tool name is untrusted data. Unknown names must be rejected without dynamic evaluation or dispatch to arbitrary code.

8.2.2 JSON syntax is only the first gate

Successful parsing establishes a data structure, not its meaning.

Worked reasoning. Allowed: tool=mean with finite values. Not allowed: tool=delete_files even when the JSON parses.

Check. Can a structured-output guarantee replace permissions?

Explanation and boundary. No. Syntax, schema conformance, semantics and authorization are distinct guarantees.

8.2.3 Use exact fields and explicit types

Reject booleans, NaN, empty arrays and oversized arrays before calculation.

Worked reasoning. In Python isinstance(True,int) is true; the validator uses type(x) in (int,float) and a finite check.

Check. Why reject a Boolean in numeric readings?

Explanation and boundary. Its programming-language subtype does not establish the intended measurement meaning. The declared contract accepts numeric readings only.

8.2.4 No natural-language sentence grants tools

A retrieved instruction remains content, even when it looks urgent.

Worked reasoning. Try delete_files and a values array containing 'ignore rules'. Verify no external side effect occurs.

Check. What is the enforcement point?

Explanation and boundary. The allowlist and schema validator, not a promise in the system prompt. These probes cover only the supplied implementation and cases.

8.2.5 Numerical results need their own checks

Valid inputs can still overflow a calculation; validate the result too.

Worked reasoning. Test the bounded mean implementation with large finite inputs; it must return a finite result or an explicit rejection.

Check. Does a valid schema prove the arithmetic is correct?

Explanation and boundary. No. Independently check arithmetic against small fixtures and reject nonfinite outputs rather than silently serializing them.

8.2.6 A finite budget belongs to the harness

Count attempts before dispatch; rejected proposals still consume an attempt.

Worked reasoning. Submit three proposals. The third must be budget_exhausted and must not invoke a tool.

Check. Why count rejected attempts?

Explanation and boundary. Otherwise repeated invalid proposals can loop forever without spending the nominal tool budget.

8.2.7 Trace observable events, not hidden thoughts

Store proposal, validation status, output and stop reason.

Worked reasoning. Locate the proposal that failed and the code that blocked it. A trace is an observable execution record, not private chain-of-thought.

Check. Can a detailed trace establish truth by itself?

Explanation and boundary. No. A trace improves inspection; incorrect code can log incorrect results consistently.

8.2.8 An LLM adapter is optional, not required

Manual or captured JSON proposals exercise the same boundary at zero inference cost.

Worked reasoning. Use a well-formed and a malformed response. Record parse success, validation success and numeric correctness separately.

Check. Are we evaluating a real LLM in the core notebook?

Explanation and boundary. No. It trains a small count model and uses explicit tool proposals. Optional external outputs must be labeled with model and date and evaluated separately.

8.2.9 Vibe coding needs acceptance criteria

Ask for a small change with inputs, forbidden behavior and tests.

Worked reasoning. Add median as an allowed tool; preserve mean behavior; reject unknown tools; add odd/even/empty cases.

Check. Why is 'make this agent smarter' insufficient?

Explanation and boundary. It does not specify observable behavior or a stopping condition. Students must review actual code and preserve the reference behavior.

8.2.10 Summary: the model never owns permissions

Parsing, validation, execution and recording are separate stages.

Worked reasoning. Save a valid result, a rejection and a budget stop. Keep the original proposal alongside each event.

Check. What evidence must survive the runtime?

Explanation and boundary. Inputs, contract, source, versions, trace and exported results. A successful chat screenshot is insufficient.

8.3 H3: Tool-contract studio: test, explain, export

8.3.1 Build a proposal-to-evidence artifact

Finish eight core tasks before attempting optional changes.

Worked reasoning. Restore 5 min; core tasks 25 min; peer checks 20 min; export and exit 10 min.

Check. What do you restore after a runtime reset?

Explanation and boundary. Run the notebook top to bottom; do not paste a remembered result into an evidence record.

8.3.2 Task 1: read a learned distribution

Compare successor counts with add-one probabilities for one context character.

Worked reasoning. With vocabulary size V and context count N, add-one assigns $1/(N+V)$ to an unseen successor.

Check. What did smoothing assume?

Explanation and boundary. A symmetric pseudo-count over the vocabulary. It avoids zero probability but is not evidence that the successor was observed.

8.3.3 Task 2: compare sampling controls

Record entropy for the same logits at two positive temperatures.

Worked reasoning. Use a fixed seed per run and preserve the probability vectors, not just the sampled strings.

Check. Can two different temperatures produce the same short string?

Explanation and boundary. Yes. One sampled continuation is a noisy observation and does not identify the underlying distribution.

8.3.4 Task 3: reject the unknown capability

Send an unknown tool name to the same dispatcher.

Worked reasoning. Record expected status before calling the dispatcher. Verify that no accepted result is produced.

Check. Should the validator silently rename an unknown tool?

Explanation and boundary. No. Silent repair changes the requested operation. Return a specific failure and require an explicit new proposal.

8.3.5 Task 4: classify malformed readings

Test empty, Boolean, string, NaN and infinite values separately.

Worked reasoning. The JSON parser rejects nonstandard constants; numeric validation also rejects nonfinite native Python values.

Check. Why keep separate tests for text and Python objects?

Explanation and boundary. They enter through different boundaries. Passing one path does not prove the other path uses the same checks.

8.3.6 Task 5: stop a repeated proposal loop

A cap counts every attempt, not only successful actions.

Worked reasoning. A cap of two permits at most two validations/dispatch attempts; the third event records a budget stop.

Check. What would happen if only successes counted?

Explanation and boundary. A sequence of malformed proposals could make no progress indefinitely while never reaching the cap.

8.3.7 Task 6: repair syntax without changing intent

Correct malformed JSON manually and rerun the exact same contract.

Worked reasoning. Repair a trailing comma. Explain why the fixed string still needs semantic validation.

Check. Does syntax repair justify executing a newly added tool?

Explanation and boundary. No. A repair may change meaning. Compare fields and retain the allowlist regardless of parse success.

8.3.8 Task 7: expose arithmetic with a fixture

Check mean([1,2,6]) = 3 before reading a model explanation.

Worked reasoning. A zero-mean fixture can hide a wrong denominator; [1,2,6] makes the expected result informative.

Check. What does one passing fixture establish?

Explanation and boundary. Agreement on that fixture only. Add empty, large, negative and length-bound cases before broader claims.

8.3.9 Task 8: hand over the evidence ZIP

Export eight predictions, eight explanations and three reflections.

Worked reasoning. Blank answers yield draft status. Filled fields mean ready for discussion, not automatically correct.

Check. What should a partner inspect first?

Explanation and boundary. Read the contract and code before running it; compare actual results with independently calculated fixtures.

8.3.10 Summary: extend only a declared boundary

Optional A adds median; optional B tests proposal formatting variants.

Worked reasoning. Explain one observation for each concept and one limitation. Keep the original reference unchanged.

Check. Why are these three different questions?

Explanation and boundary. Token likelihood describes generation, permission controls actions, and correctness requires task evidence. None implies the others.

8.4 Mathematical reference

$$p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{<t}), \qquad L = -\frac{1}{T}\sum_{t=1}^{T} \log p(x_t \mid x_{<t}).$$

$$P_{ij} = \frac{C_{ij} + 1}{\sum_k C_{ik} + V}, \qquad p_i(T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}, \quad T > 0.$$

The count model uses one-character context, not Transformer attention. Likelihood and entropy are not measures of factual support.

8.5 Eight core practice tasks

8.5.1 Task 1: Inspect a learned next-character distribution

Predict the most frequent successor of a space. Read the actual transition counts, then compare the normalized add-one probabilities.

Self-check. All successor probabilities, not just the six plotted, sum to one. A smoothed nonzero probability does not imply an observed transition.

8.5.2 Task 2: Change temperature without changing the model

Predict how T=0.5 and T=2 change entropy. The corpus and trained counts stay fixed; compare distributions as well as generated samples.

Self-check. T must be finite and positive. Entropy concentration is not a factual-accuracy measure.

8.5.3 Task 3: Reject an unknown tool before execution

Predict statuses for a mean proposal and a forbidden capability. No files, network requests or real messages are touched.

Self-check. The forbidden proposal produces no numeric result. No eval, shell tool or dynamic import is used to dispatch a name.

8.5.4 Task 4: Probe malformed numeric inputs

Predict a reason for each rejection before running. Python values and text JSON use different entrance gates.

Self-check. All six malformed native inputs reject. Nonstandard JSON NaN is also rejected by the parser in Task 6.

8.5.5 Task 5: Exhaust the attempt budget

Predict the third status when two proposals have already consumed the two-attempt cap. Rejected proposals consume attempts too.

Self-check. Only two proposals enter dispatch. The third is a stop event, not a third tool attempt.

8.5.6 Task 6: Repair syntax while preserving the boundary

Compare a valid proposal, a trailing comma, a duplicate key and NaN text. Repair only the syntax of a copied string.

Self-check. Duplicate fields are not silently overwritten. Syntax repair still passes through the same validator.

8.5.7 Task 7: Independently check arithmetic

Calculate 1+2+6 by hand, divide by 3, then check the tool. Explain why zero-mean inputs might hide a denominator bug.

Self-check. The expected 3.0 was written independently, not generated by dispatch. One fixture is not a full arithmetic evaluation.

8.5.8 Task 8: Audit what the exported record can support

Check every task is present, preserve a corpus hash and inspect one rejection with its original proposal.

Self-check. The final export records incomplete reflections as draft. No model account or paid quota is needed.

8.6 Evidence handover and summary

Explain next-token prediction; validate structured tool proposals; retain control outside the model. Save eight predictions, eight explanations and three reflections. Preserve source, data, versions, traces and one limitation. The notebook exports a draft when fields are blank; response completeness does not establish correctness. Save the edited notebook separately if changing code.

8.7 Primary references

- Attention Is All You Need: https://arxiv.org/abs/1706.03762.
- Building effective agents: https://www.anthropic.com/engineering/building-effective-agents.

Chapter 9

Retrieval, source evidence, and structured knowledge

Week 13: 2026-11-30, Monday 14:00–17:00.

Use original fictional lab passages and independently labeled questions. This lexical retrieval and extractive evidence workflow does not call a generative model; it is a baseline for, not a full implementation of, RAG.

Learning outcomes

Build a lexical retrieval baseline; measure retrieval separately from grounded claim support; abstain without evidence.

The meeting contains three 60-minute modules: 60 minutes explanation and 120 minutes practice. The eight core exercises and two optional challenges are in-class practice, not additional weekly graded homework.

9.1 H1: Represent documents and retrieve evidence

9.1.1 Retrieval selects evidence, not truth

Find relevant passages before composing a source-bounded response.

Worked reasoning. Document identity; lexical vectors; cosine similarity; ranking; threshold; Task 1 and Task 2.

Check. Can the best-ranked passage be wrong?

Explanation and boundary. Yes. Ranking measures a chosen similarity, not the authority or factual correctness of a source.

9.1.2 Document identity must survive chunking

Store source ID, version and chunk location with every passage.

Worked reasoning. LAB-02:v1:chunk0 identifies an original teaching protocol passage; it is not an external scientific publication.

Check. What breaks when IDs are just row positions?

Explanation and boundary. Inserting a passage changes positional identities. Use stable source keys and preserve the indexed text snapshot.

9.1.3 Tokenization defines lexical overlap

A vocabulary maps terms to coordinates; preprocessing changes what matches.

Worked reasoning. Vocabulary [heat,limit,mean]; document 'heat heat limit' maps to [2,1,0].

Check. Does this representation preserve word order?

Explanation and boundary. No. Bag-of-words counts lose order; negation and relational meaning can be missed.

9.1.4 IDF discounts corpus-common terms

Smoothed IDF is ln((1+N)/(1+df))+1 in this notebook.

Worked reasoning. N=4, df=1 gives ln(5/2)+1=1.916; df=4 gives 1.0.

Check. Is a rare term necessarily an important fact?

Explanation and boundary. No. IDF is a statistical corpus weight. Rare typos and irrelevant words can also receive large weights.

9.1.5 TF-IDF is a fitted representation

Fit the vectorizer on the frozen corpus; transform questions with it.

Worked reasoning. The same query should be scored against one fixed index. New documents require an explicit index version.

Check. What happens to unseen query words?

Explanation and boundary. They do not create new fitted coordinates. A fully out-of-vocabulary query gives a zero vector under this baseline.

9.1.6 Cosine compares directions

Normalize vectors before using their dot product as similarity.

Worked reasoning. q=[1,1], d=[2,0]; cosine=2/(sqrt(2)*2)=0.707.

Check. Is cosine a probability that the passage is correct?

Explanation and boundary. No. It is a geometric similarity under the representation. A threshold is an operating rule, not calibrated confidence.

9.1.7 Top-k is a selection rule

Rank descending; break equal-score ties with stable source IDs.

Worked reasoning. If scores are equal, sort document IDs lexically so the same snapshot returns the same order.

Check. Can a deterministic tie rule improve relevance?

Explanation and boundary. Not inherently. It improves reproducibility and makes ambiguous rankings visible.

9.1.8 Thresholds trade coverage and abstention

Return no evidence when the maximum score is below a declared threshold.

Worked reasoning. Use an in-vocabulary protocol question and an unrelated vocabulary question. Inspect full score vectors.

Check. Does a low score prove the answer does not exist?

Explanation and boundary. No. Lexical mismatch can hide a relevant passage. Abstention is a limitation-aware action, not a proof of absence.

9.1.9 Chunking can remove required context

A passage without its units or exception can support the wrong claim.

Worked reasoning. Compare 'review if mean at least 80 C' with a chunk containing only 'review if mean at least 80'.

Check. Why is the shorter chunk risky?

Explanation and boundary. The missing unit changes interpretation, and retrieval alone cannot recover a qualifier absent from the supplied passage.

9.1.10 Summary: freeze the representation

Preserve corpus, source IDs, vectorizer configuration and ranking rule.

Worked reasoning. Show the exact quote, score and source version. Explain one lexical retrieval limitation.

Check. What will the next hour add?

Explanation and boundary. Independent relevance labels and claim-support checks. A retrieved citation is not yet a verified answer.

9.2 H2: Measure retrieval and check claim support

9.2.1 Two evaluation layers answer different questions

Retrieval asks whether evidence was found; grounding asks whether a claim follows.

Worked reasoning. A relevant passage can be retrieved while the answer misstates its threshold.

Check. Can perfect retrieval recall guarantee a correct response?

Explanation and boundary. No. Generation, interpretation and citation entailment can fail after retrieval succeeds.

9.2.2 Recall at k needs a relevance set

Recall@k = retrieved relevant sources / all labeled relevant sources.

Worked reasoning. Relevant={A,B}; top2={A,C}; recall@2=1/2. Precision@2=1/2.

Check. What if no relevant source is labeled?

Explanation and boundary. Do not divide by zero or fabricate relevance. Evaluate no-evidence/abstention cases separately.

9.2.3 Reciprocal rank rewards the first hit

For an answerable query, RR = 1/rank of the first relevant result.

Worked reasoning. [C,A,B] and [A,C,B] may have the same recall@3 but different reciprocal rank.

Check. Why report the rank convention?

Explanation and boundary. Metrics depend on k, relevance labels, tie rules and zero-hit handling. Preserve these definitions with the result.

9.2.4 An evidence quote has a narrow role

An exact substring proves text location, not whether every derived claim follows.

Worked reasoning. Source: threshold 80 C. Claim: threshold 95 C. A correct source ID does not rescue the wrong number.

Check. What does automatic quote containment miss?

Explanation and boundary. It misses paraphrase meaning, omitted exceptions and contradictions. Human or task-specific semantic checks remain necessary.

9.2.5 Task 3: score fixed retrieval questions

Use independently written relevance IDs; do not derive labels from ranking.

Worked reasoning. Compute macro recall only across answerable queries; list no-evidence outcomes separately.

Check. Why not label the top result as relevant?

Explanation and boundary. That would make the system grade itself and conceal retrieval failures.

9.2.6 Task 4: inspect false support

Test wrong citation IDs, fabricated quotes and changed numeric thresholds.

Worked reasoning. Unknown source; absent quote; quoted text with a contradictory threshold. They require different diagnoses.

Check. Will string checks catch every wrong claim?

Explanation and boundary. No. They are necessary structural checks in this exercise, not a universal entailment detector.

9.2.7 Task 5: change k under a fixed index

Keep questions, labels and threshold fixed while varying k.

Worked reasoning. Recall cannot decrease when adding ranked results under the same accepted query; precision can decrease.

Check. Why can a threshold obscure that comparison?

Explanation and boundary. A changed threshold changes which results are eligible. Keep it fixed and report abstention separately.

9.2.8 Treat source instructions as data

A malicious sentence in a passage cannot alter tool permissions.

Worked reasoning. The corpus includes an explicitly marked untrusted instruction. Retrieval may return it, but the workflow never executes it.

Check. Does a low similarity score secure a RAG system?

Explanation and boundary. No. Relevant malicious text can rank highly. Security enforcement must not depend on the retriever missing it.

9.2.9 Structured knowledge needs provenance too

A table row should retain units, source ID, version and extraction status.

Worked reasoning. name=review threshold; value=80; unit=C; source=LAB-02:v1; status=teaching fixture.

Check. Why not call this a validated engineering limit?

Explanation and boundary. It is an original fictional class protocol. It has no external safety authority or hardware validation.

9.2.10 Summary: evidence retrieval precedes explanation

Find a passage, preserve its identity, then check what it actually supports.

Worked reasoning. Save ranked IDs, scores, labels and one support failure. Do not export a bare average.

Check. What should a partner be able to reconstruct?

Explanation and boundary. Which question, index snapshot, ranking and relevance labels produced each reported metric.

9.3 H3: Retrieval studio: diagnose and package failures

9.3.1 Build an evidence-first retrieval record

Complete eight core tasks, then preserve an honest failure.

Worked reasoning. Restore 5 min; tasks 25 min; source audit 15 min; export and explain-back 15 min.

Check. What is the artifact, not just the score?

Explanation and boundary. A corpus snapshot, ranking configuration, per-query records, response fields, figures and hashes.

9.3.2 Task 1: distinguish answerable and unknown

Inspect the maximum cosine and abstention status for both questions.

Worked reasoning. The system returns no evidence instead of inventing a threshold or citation.

Check. What does the abstention support?

Explanation and boundary. Only that this index and operating rule supplied no accepted passage for this query.

9.3.3 Task 2: explain one weighted term

Compare term frequency, document frequency and IDF contribution.

Worked reasoning. An IDF of 1.0 is not zero; the default smoothed formula retains corpus-common terms.

Check. What changes if the corpus changes?

Explanation and boundary. Vocabulary, document frequencies, IDF, norms and rankings may all change. Preserve an index version.

9.3.4 Task 3: compute macro retrieval recall

Calculate per-query recall before averaging answerable cases.

Worked reasoning. Two queries with one and two relevant documents each receive equal weight under macro averaging.

Check. Is macro recall the same as pooled recall?

Explanation and boundary. Not generally. Pooled recall weights by the number of relevant documents. Name the aggregation you use.

9.3.5 Task 4: find a citation failure

Compare a truthful quote with a wrong-threshold claim.

Worked reasoning. Do not say only 'bad citation'. Identify the changed number, missing unit or absent quote.

Check. Can an automatic pass replace source reading?

Explanation and boundary. No. The class checks exact fields and narrow numeric facts; general semantic support remains a separate review.

9.3.6 Task 5: vary k without tuning on the final set

Use development questions for exploration; preserve held-out questions.

Worked reasoning. Record k and threshold before examining the held-out results. If already seen, label further work exploratory.

Check. What makes a final report optimistic?

Explanation and boundary. Repeatedly changing retrieval settings after seeing held-out outcomes and reporting only the best score.

9.3.7 Task 6: add a genuinely independent query

Write its relevant IDs before execution and justify them from source text.

Worked reasoning. Try a paraphrase to expose lexical mismatch. Explain whether a miss is due to wording or missing corpus evidence.

Check. Is one new query a representative benchmark?

Explanation and boundary. No. It is a diagnostic probe. Broader coverage requires a task-based question set.

9.3.8 Task 7: separate tool results from passages

A computed mean is not a retrieved protocol threshold.

Worked reasoning. Mean 81 C is computed from readings; threshold 80 C comes from the fictional protocol; comparison requests review.

Check. What remains outside this exercise?

Explanation and boundary. Physical measurement validity, real safety limits, automatic control authority and external deployment validation.

9.3.9 Task 8: export source-bounded evidence

Keep corpus hash and exact retrieved text with the metric table.

Worked reasoning. They select a query, locate its IDs and quotes, and reproduce its recall from the labeled relevant set.

Check. Why preserve text as well as a link?

Explanation and boundary. Sources and chunks can change. A frozen snapshot makes the reported computation inspectable.

9.3.10 Summary: ask what the source entails

Optional A changes chunking; optional B adds an adversarial passage.

Worked reasoning. Explain each outcome using a specific source or absent source, not a vague model-confidence story.

Check. What connects to the next week?

Explanation and boundary. The same untrusted evidence must pass through explicit permissions, budgets and approval gates before any action.

9.4 Mathematical reference

$$\mathrm{idf}(t) = \log \frac{1+N}{1+\mathrm{df}(t)} + 1, \qquad \cos(q,d) = \frac{q^\top d}{\|q\|\|d\|}.$$

$$\text{Recall@k} = \frac{|R \cap K|}{|R|}, \quad |R| > 0, \qquad \mathrm{RR} = \frac{1}{\text{first relevant rank}}.$$

Handle zero relevant sources as a separate no-evidence evaluation, not a zero-denominator recall. Lexical similarity is not calibrated confidence.

9.5 Eight core practice tasks

9.5.1 Task 1: Compare relevant evidence and abstention

Predict the returned source for a unit question and the behavior for an out-of-vocabulary question.

Self-check. The unknown query has no accepted passage. Similarity values are not calibrated factual-confidence probabilities.

9.5.2 Task 2: Inspect the fitted lexical representation

Predict whether a rarer word receives larger IDF. Then inspect actual vocabulary and normalized document coordinates.

Self-check. All six nonzero document vectors have L2 norm 1. Corpus-common terms still have nonzero smoothed IDF.

9.5.3 Task 3: Evaluate fixed held-out relevance labels

Freeze k=2 and threshold=0.12 before this cell. Predict a source for each held-out question and report no-evidence cases separately.

Self-check. Do not average an undefined recall for the no-evidence query as zero. After seeing this set, later tuning is exploratory.

9.5.4 Task 4: Separate citation structure from claim support

Predict which failure each answer should trigger. Exact-quote checking is only one layer of support.

Self-check. Even the first answer still requires semantic review. The checker handles a narrow numeric fixture, not general entailment.

9.5.5 Task 5: Change k on development questions only

Compare k=1 and k=2 with the same fitted corpus and threshold. Keep held-out results fixed and name this as development exploration.

Self-check. Recall cannot decrease when an unchanged accepted ranked list is extended. This is not permission to tune on already inspected held-out outcomes.

9.5.6 Task 6: Add one independently labeled diagnostic query

Write a paraphrase and relevant IDs before retrieval. The supplied example is a diagnostic, not a representative new benchmark.

Self-check. A lexical miss is useful evidence. Do not silently relabel a missed source as irrelevant after seeing the ranking.

9.5.7 Task 7: Combine a numeric result and a source rule

Compute the mean independently, retrieve the class threshold passage and keep their provenance separate.

Self-check. The value 81 is computed, while the threshold 80 is source-derived. Neither establishes a real equipment limit.

9.5.8 Task 8: Preserve corpus identity and one failure

Export ranked IDs, exact text, fixed labels and one unsupported claim. The source snapshot is part of the evidence.

Self-check. Source IDs and exact quotes are provenance anchors, not a guarantee that the source is true.

9.6 Evidence handover and summary

Build a lexical retrieval baseline; measure retrieval separately from grounded claim support; abstain without evidence. Save eight predictions, eight explanations and three reflections. Preserve source, data, versions, traces and one limitation. The notebook exports a draft when fields are blank; response completeness does not establish correctness. Save the edited notebook separately if changing code.

9.7 Primary references

- Retrieval-Augmented Generation: https://arxiv.org/abs/2005.11401.
- TfidfVectorizer reference: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.

Chapter 10

Agent testing, failure cases, and oversight

Week 14: 2026-12-07, Monday 14:00–17:00.

Test a deterministic local state machine. All messages are append-only Python list events to a fictional recipient; no mail, files, model inference or external side effect is invoked.

Learning outcomes

Enforce bounded execution; test authorization and failures; bind simulated approvals to exact actions.

The meeting contains three 60-minute modules: 60 minutes explanation and 120 minutes practice. The eight core exercises and two optional challenges are in-class practice, not additional weekly graded homework.

10.1 H1: State, budgets, and trust boundaries

10.1.1 An agent is a controlled loop

An action proposal must pass the harness before changing the environment.

Worked reasoning. State; transition; attempt budget; tool permissions; tool errors; approvals; trace evidence.

Check. Where does the model lose authority?

Explanation and boundary. At every deterministic gate. A proposal cannot change the action set or enlarge its own budget.

10.1.2 Workflow and agent are not synonyms

A fixed graph is a workflow; model-directed choices add an agent decision layer.

Worked reasoning. A three-step measurement report can use a script. Do not add open-ended planning when the steps are known.

Check. Does a loop alone make a system an LLM agent?

Explanation and boundary. No. The core notebook is a deterministic harness simulator. The language-model decision component is deliberately absent.

10.1.3 Represent state before writing actions

State records attempts, observations, pending approval and terminal status.

Worked reasoning. After budget_exhausted, later proposals must not execute. A terminal status is not just a displayed message.

Check. What is the invariant after stopping?

Explanation and boundary. The executed action count and side-effect log do not change. Tests should attempt an additional action.

10.1.4 Budgets need an accounting unit

Count attempts, tool calls and declared work separately.

Worked reasoning. Three invalid proposals under a two-attempt cap must stop at the cap even though no tool ran.

Check. Can a tool-call cap guarantee short runtime?

Explanation and boundary. No. One call may hang or be expensive. Wall-clock limits and resource isolation are distinct controls.

10.1.5 Validate before dispatch

Unknown tool names must not reach an implementation or external service.

Worked reasoning. Compare a permitted mean request with a fake send_email request. The simulator sends no real message.

Check. What must the trace prove about the forbidden request?

Explanation and boundary. It was rejected before dispatch; do not rely on a downstream tool happening to fail.

10.1.6 Use typed errors with narrow recovery

Invalid input, transient tool failure and permission failure need different next steps.

Worked reasoning. A bounded retry can handle a simulated transient error; it cannot authorize a forbidden operation.

Check. Why can retries make behavior worse?

Explanation and boundary. Repeated side effects, cost and duplicate operations may occur. Retry only a declared safe operation with a bound.

10.1.7 Untrusted text cannot amend policy

Retrieved instructions are data even if they imitate a system message.

Worked reasoning. Store the fixture text and show that the allowed tools remain unchanged after it is observed.

Check. Does this test prove universal prompt-injection resistance?

Explanation and boundary. No. It tests one deterministic boundary and supplied cases; real model-mediated paths require broader adversarial evaluation.

10.1.8 Approval must name the exact action

Bind action, arguments and a unique nonce into a reviewed digest.

Worked reasoning. An approval for draft A cannot be reused for draft B with different content or destination.

Check. Why is an 'approved=true' field insufficient?

Explanation and boundary. Untrusted callers could set it themselves. Approval must come from a separate authority and be checked against the exact request.

10.1.9 Consumption prevents approval replay

A valid approval is one-use and has a declared expiration rule.

Worked reasoning. The first simulated action can succeed; the second must be rejected without a duplicate event.

Check. What limitation remains in this teaching approval?

Explanation and boundary. It models a local trusted registry, not distributed identity, cryptographic authentication, real messaging or production security.

10.1.10 Summary: make unsafe transitions impossible

Place gates in code rather than hoping a model follows instructions.

Worked reasoning. Show why a retrieved sentence cannot reach the simulated send action without a separately approved digest.

Check. What will the next hour measure?

Explanation and boundary. Expected versus actual behavior for each boundary, including errors, stops and approval misuse.

10.2 H2: Failure cases and human approval tests

10.2.1 A safety claim needs a negative test

A normal success case does not probe a forbidden transition.

Worked reasoning. Each row states initial state, proposal, expected status and permitted side effects.

Check. Why not use only example conversations?

Explanation and boundary. They may omit the actual execution boundary. Test the harness state and side-effect log directly.

10.2.2 Test status and side effects together

A rejection message is insufficient if the operation already happened.

Worked reasoning. A function can append an event and then return rejected. Checking only the string would miss the violation.

Check. Which assertion detects that bug?

Explanation and boundary. Assert that the event count and protected state are unchanged for every rejected action.

10.2.3 Task 4: probe schema edge cases

Boolean, NaN, empty values and unknown fields should fail explicitly.

Worked reasoning. A separate case reveals which check was skipped when a regression occurs.

Check. Does rejecting all requests make a good agent?

Explanation and boundary. No. Include legitimate requests to measure functionality alongside safety. Always rejecting can pass a narrow negative-only suite.

10.2.4 Task 5: exhaust the attempt cap

Use a plan longer than the cap and inspect the exact stop index.

Worked reasoning. The simulator records two consumed attempts at cap two; a later proposal returns stopped.

Check. Why test after the stop?

Explanation and boundary. It verifies terminal-state enforcement rather than merely detecting that a stop message once appeared.

10.2.5 Task 6: bind approval to arguments

Change one field after approval and verify digest mismatch.

Worked reasoning. Wrong nonce, expired approval and already-consumed approval are distinct rejection cases.

Check. Can a digest prove who approved it?

Explanation and boundary. No. It detects request mismatch relative to a trusted approval record. Identity assurance is outside this local simulator.

10.2.6 Simulated timeout is not process isolation

A fake slow-step flag exercises an error branch, not operating-system enforcement.

Worked reasoning. The notebook has no blocking network tool. A production tool requires real timeout and cancellation controls.

Check. Why avoid claiming runtime security here?

Explanation and boundary. An in-process check cannot forcibly stop arbitrary blocking code. The demonstration only tests control-flow handling.

10.2.7 Retry only a declared transient failure

A bounded retry preserves the original request and logs each attempt.

Worked reasoning. Timeout may be retryable for a read; permission_denied is never repaired by repeated calls.

Check. What if a request has a side effect?

Explanation and boundary. A retry can duplicate it. Use idempotency keys or transactional semantics, not blind re-execution.

10.2.8 Audit traces are bounded evidence

Record events without secrets, irrelevant personal data or fabricated reasoning.

Worked reasoning. Identify the first failed precondition and show that later protected events are absent.

Check. Does a trace prove the implementation was not bypassed?

Explanation and boundary. Not by itself. Tests, provenance and independent observation of the execution boundary are also needed.

10.2.9 Human review must offer a real choice

Show exact action, evidence and consequences before approval.

Worked reasoning. Name recipient, content, reason and whether the operation is reversible. Our notebook uses local fictional destinations only.

Check. Why is vague confirmation weak?

Explanation and boundary. A user cannot meaningfully authorize an unspecified or changed operation. Review scope must match execution scope.

10.2.10 Summary: safety and usefulness are separate

Report legitimate success and forbidden-action violations with denominators.

Worked reasoning. Save per-case expected/observed statuses and event counts, not just 'all passed'.

Check. What does zero observed violations mean?

Explanation and boundary. No violation was observed in this finite suite. It is not a universal guarantee of safe deployment.

10.3 H3: Oversight studio: attack the boundary

10.3.1 Probe the harness without real side effects

All destructive, messaging and approval operations remain simulated.

Worked reasoning. Restore 5 min; core tests 25 min; approval review 15 min; peer audit and export 15 min.

Check. What is not being tested?

Explanation and boundary. Real identity services, real mail APIs, operating-system sandboxes and a live language model.

10.3.2 Task 1: accept a legitimate operation

Verify the permitted mean and its independent expected value.

Worked reasoning. A dispatcher that rejects every proposal is not a useful reference even if it never performs a forbidden action.

Check. Which denominator belongs to legitimate success?

Explanation and boundary. The number of valid requests in the declared suite, not all requests including deliberate attacks.

10.3.3 Task 2: keep malicious text powerless

Pass the injection fixture as content, never as executable instructions.

Worked reasoning. The allowed-tool tuple and approval registry must not be modified by retrieved prose.

Check. What would invalidate the test?

Explanation and boundary. Changing the contract in the test to make the attack pass or executing a parsed instruction from the hostile text.

10.3.4 Task 3: prove one-use approval

Inspect the digest and event count after the first use and replay.

Worked reasoning. A replay test needs the same registry. A fresh registry would represent a different authorization context.

Check. Why does this matter for reproducibility?

Explanation and boundary. Stateful security behavior depends on event order. Preserve initial state and the exact sequence of requests.

10.3.5 Task 4: differentiate malformed inputs

Name the failing precondition for each deliberately invalid proposal.

Worked reasoning. For True, explain numeric subtype confusion; for NaN, explain nonfinite arithmetic and JSON portability.

Check. Why not catch every exception and return success?

Explanation and boundary. It would erase the failure semantics and could hide partial execution or implementation errors.

10.3.6 Task 5: inspect the terminal state

After cap exhaustion, compare the complete action log before and after.

Worked reasoning. Reset only by an explicit new session with a fresh state record and a new evidence ID.

Check. When is a reset legitimate?

Explanation and boundary. When declared as a new run, not as an invisible way to exceed the budget of an ongoing task.

10.3.7 Task 6: modify an approved request

Change one argument without issuing a new approval.

Worked reasoning. A partner should see exactly which field changed and why the original approval no longer matches.

Check. Could the caller approve its own changed request?

Explanation and boundary. Not under this contract. Approval creation is a separate trusted teaching action, not an allowed model tool.

10.3.8 Task 7: reconstruct the first failure

Follow the trace from proposal to rejection to stop.

Worked reasoning. Explain the observed predicate and result. Avoid inventing what a model internally thought.

Check. What is a defensible explanation?

Explanation and boundary. A mechanism tied to code, inputs and trace events with a stated boundary on what was observed.

10.3.9 Task 8: preserve a negative result

Export a case that fails as expected and one valid success.

Worked reasoning. The expected error must be caught by the test, not hidden by removing assertions.

Check. When does an expected rejection become a defect?

Explanation and boundary. When a valid request is rejected or an invalid request causes an unauthorized side effect.

10.3.10 Summary: extend one threat model at a time

Optional A adds a bounded retry; optional B alters one approved argument.

Worked reasoning. Name a real-world boundary that the local simulator cannot establish and propose a suitable next test.

Check. What connects to Studio I?

Explanation and boundary. The workflow will combine retrieval and numerical evidence while preserving these same permission and stop boundaries.

10.4 Invariant reference

$$0 \leq \text{consumed attempts} \leq \text{cap}.$$

A stopped session cannot execute another action. Approval must match the exact action and nonce, remain unexpired, and be unused. A digest binds bytes relative to a trusted registry; it does not establish the identity of the reviewer. Simulated timeout branches do not enforce real wall-clock cancellation.

10.5 Eight core practice tasks

10.5.1 Task 1: Accept a valid allowed calculation

Predict a valid result and verify that the permitted action is useful, not just always rejected.

Self-check. Legitimate task success is different from negative-case rejection correctness.

10.5.2 Task 2: Keep hostile source text outside authority

Record the hostile text as data and compare the action boundary before and after.

Self-check. This probes a fixed deterministic allowlist. It does not measure live LLM prompt-injection robustness.

10.5.3 Task 3: Use approval once and reject replay

The trusted reviewer binds one fictional recipient, text and nonce. Reuse the same harness for both calls.

Self-check. Exactly one list event is appended. This is a trusted local registry simulation, not cryptographic authentication.

10.5.4 Task 4: Probe data boundaries and side effects

Create a fresh harness per malformed fixture so the budget does not hide individual validation checks.

Self-check. Expected errors are caught and asserted. They are not removed to force a successful notebook run.

10.5.5 Task 5: Prove a terminal budget state

A cap of two counts invalid attempts. Probe a fourth proposal after the budget stop.

Self-check. A stopped session cannot execute a later valid proposal. Only an explicit new session can reset the budget.

10.5.6 Task 6: Reject changed or expired approvals

Compare exact-action mismatch and expiry in separate fresh harnesses; neither may append an event.

Self-check. Expiry uses an explicit simulated time input. Real clocks, identity systems and network races are outside scope.

10.5.7 Task 7: Read the first failed invariant

Construct a bounded mixed trace. Diagnose the first failure from observable predicates rather than hidden model reasoning.

Self-check. The first failed precondition is the empty numeric list. The later unapproved message never creates an event.

10.5.8 Task 8: Separate finite-suite evidence and broad claims

Record the tested invariants and explicitly name the real boundaries that were not exercised.

Self-check. Zero observed unauthorized events in a finite simulator suite is not a universal safety guarantee.

10.6 Evidence handover and summary

Enforce bounded execution; test authorization and failures; bind simulated approvals to exact actions. Save eight predictions, eight explanations and three reflections. Preserve source, data, versions, traces and one limitation. The notebook exports a draft when fields are blank; response completeness does not establish correctness. Save the edited notebook separately if changing code.

10.7 Primary references

- Building effective agents: https://www.anthropic.com/engineering/building-effective-agents.
- NIST Generative AI Profile: https://doi.org/10.6028/NIST.AI.600-1.

Chapter 11

Agent Studio I: build a bounded workflow

Week 15: 2026-12-14, Monday 14:00–17:00.

Implement a deterministic engineering-report baseline using original synthetic temperature readings and a fictional review protocol. No physical diagnosis, equipment command or live LLM inference is performed.

Learning outcomes

Build a source-bounded thermal-report workflow from synthetic readings with explicit units, checks and review status.

The meeting contains three 60-minute modules: 60 minutes explanation and 120 minutes practice. The eight core exercises and two optional challenges are in-class practice, not additional weekly graded homework.

11.1 H1: Specify an engineering evidence workflow

11.1.1 Build a report, not a physical controller

The system summarizes synthetic readings and requests review; it never controls equipment.

Worked reasoning. Measurement meaning; unit conversion; repeated readings; uncertainty; threshold; source provenance.

Check. What does 'review required' mean here?

Explanation and boundary. A fictional class protocol requests human review. It is not a hardware safety diagnosis or a command to act.

11.1.2 A row needs physical meaning

Our readings are repeated temperature observations of one synthetic bench case.

Worked reasoning. Five repeated readings belong to one fictional case, not five independent deployment trials.

Check. Why does row meaning affect evaluation?

Explanation and boundary. It determines the unit of analysis and whether a reported sample size counts cases, readings or correlated repeats.

11.1.3 Units are part of the input type

Accept C or F explicitly; convert before comparing a Celsius threshold.

Worked reasoning. 176 F becomes (176-32)*5/9 = 80 C. Mean 176 F is not compared directly with 80 C.

Check. What should happen to an unknown unit?

Explanation and boundary. Reject it with a specific error. Never infer units from a plausible numeric range.

11.1.4 Summaries answer different questions

Mean describes location; sample standard deviation describes observed spread.

Worked reasoning. [78,80,82] C has mean 80 C and sample SD 2 C, using denominator n-1.

Check. Is sample SD an uncertainty guarantee?

Explanation and boundary. No. It summarizes observed spread; calibration, dependence, drift and measurement uncertainty require additional evidence.

11.1.5 Missing and nonfinite data require a policy

Reject missing, Boolean, infinite and NaN readings in this reference.

Worked reasoning. A missing reading does not become zero, and dropping it changes the sample definition.

Check. Why require at least two readings?

Explanation and boundary. The sample SD with denominator n-1 is undefined for one observation. A different contract could return an explicit unavailable spread.

11.1.6 The threshold comes from a source

A versioned fictional teaching protocol sets review if mean >=80 C.

Worked reasoning. Mean exactly 80 C must request review under the declared inclusive rule.

Check. Is this a certified safety limit?

Explanation and boundary. No. It is a transparent original teaching fixture with no real equipment authority.

11.1.7 Predeclare behavior for disagreement

If required source evidence is absent, abstain rather than invent a threshold.

Worked reasoning. Mean 81 C without the source threshold does not justify a protocol-based review status.

Check. Can a model fill the missing protocol from memory?

Explanation and boundary. Not under this evidence contract. Ask for the approved source snapshot.

11.1.8 Separate data errors from review decisions

A valid high mean can yield review; invalid readings yield rejection.

Worked reasoning. Valid low mean, valid equality mean and missing unit must have different statuses.

Check. Why is this distinction important?

Explanation and boundary. A data-quality failure cannot be presented as a successful numerical or engineering conclusion.

11.1.9 Make the report schema explicit

Return case ID, input hash, mean C, sample SD C, source and status.

Worked reasoning. Locate the exact source ID and input digest before reading the prose summary.

Check. Which field proves measurement authenticity?

Explanation and boundary. None. A digest preserves byte identity relative to a reference, not instrument authenticity or physical validity.

11.1.10 Summary: define meaning before automation

Units, sample definition, protocol and forbidden actions precede code generation.

Worked reasoning. Explain why a single reading is rejected and why equality triggers review.

Check. What does the next hour implement?

Explanation and boundary. A deterministic workflow and trace. An optional LLM can propose structured inputs but cannot bypass its checks.

11.2 H2: Implement a checked analysis pipeline

11.2.1 The workflow is a graph with gates

Each stage has explicit inputs, outputs and failure behavior.

Worked reasoning. The reference uses a known graph so students can inspect each transformation without a framework.

Check. Why keep the first implementation deterministic?

Explanation and boundary. It is a clear baseline for arithmetic, units and source rules before adding uncertain language-model decisions.

11.2.2 Validate the complete request

Require exact fields and a case ID before reading numeric values.

Worked reasoning. A request containing execute_shell is invalid even if its readings are numeric.

Check. Why reject unexpected fields?

Explanation and boundary. It avoids silently accepting ambiguous capabilities. Unknown metadata must be added by an explicit contract revision.

11.2.3 Conversion needs fixture tests

Test both identity in C and a known F conversion.

Worked reasoning. [78,80,82] C and [172.4,176,179.6] F should produce the same summary within declared tolerance.

Check. Why use numeric tolerance?

Explanation and boundary. Floating-point conversion can introduce small rounding differences. Equality of decisions at the threshold also needs a stated numeric policy.

11.2.4 Preserve the threshold comparison policy

Compare converted means after rounding to 10 decimal places in this teaching workflow.

Worked reasoning. A mathematically equal Fahrenheit fixture should not fall below the threshold due only to binary rounding.

Check. Is this a universal measurement policy?

Explanation and boundary. No. It is a declared computational convention. Real tolerances should follow measurement resolution and application requirements.

11.2.5 The sample SD has a declared denominator

Use ddof=1 and reject n<2 before computing spread.

Worked reasoning. For [78,80,82], squared deviations sum to 8; divide by 2, then sqrt gives 2.

Check. Would dividing by n be equally valid here?

Explanation and boundary. It answers a different statistic. The contract explicitly requests sample SD, so a population-SD implementation is a mismatch.

11.2.6 Trace stage outputs without excessive data

Record stage, status, digest and bounded evidence.

Worked reasoning. Find validate, convert, summarize, protocol and report in order; a failure must not create later successful stages.

Check. Can trace order expose a hidden shortcut?

Explanation and boundary. It can reveal an inconsistency, but trustworthy execution also needs source inspection and independent expected outputs.

11.2.7 Abstention is a valid terminal result

Missing source returns no protocol-based review conclusion.

Worked reasoning. Keep valid readings but call with no approved source. The result must not claim review status.

Check. Why not use a default threshold?

Explanation and boundary. A default would silently invent authority absent from the declared input evidence.

11.2.8 A report must distinguish facts and limits

Numerical summary, protocol comparison and limitation belong in separate fields.

Worked reasoning. Synthetic case A has mean 80 C; fictional protocol P1 requests review at or above 80 C; no physical diagnosis follows.

Check. What is an unsupported extension?

Explanation and boundary. Claiming overheating, failure probability or a safe operating condition without relevant physical validation.

11.2.9 Test an AI-generated patch as new code

Review data flow and fixtures before replacing the reference.

Worked reasoning. The patch uses sum(values)/(n+1). Predict why a zero-mean fixture can conceal it.

Check. Why not trust plausible prose explaining the patch?

Explanation and boundary. Only source, independent expected values and execution outcomes establish its behavior on tested cases.

11.2.10 Summary: preserve the reference workflow

Save checked inputs, configuration, source, trace and result before extension.

Worked reasoning. Download the ZIP and identify one failure case. Do not overwrite the original with an experimental change.

Check. What does H3 complete?

Explanation and boundary. The eight-task evidence record, peer explanation and one optional extension while keeping the reference reproducible.

11.3 H3: Studio I: build, probe, and explain

11.3.1 Deliver one complete analysis chain

A partner must follow input, conversion, source and conclusion.

Worked reasoning. Restore 5 min; fixtures 20 min; controlled change 15 min; peer handover and export 20 min.

Check. What is the minimum useful handover?

Explanation and boundary. Runnable source plus inputs, expected fixtures, actual trace, result and an honest limitation.

11.3.2 Task 1: classify the contract outcomes

Predict low, equality, high and malformed cases before execution.

Worked reasoning. Low returns reported; equal/high return review_required; malformed returns rejected.

Check. Does reported mean physically safe?

Explanation and boundary. No. It means the fictional report pipeline completed below its class threshold. No real safety claim is authorized.

11.3.3 Task 2: audit provenance fields

Find case ID, request hash, protocol ID and implementation version.

Worked reasoning. A case ID names a record; a hash identifies canonical JSON bytes; a source ID locates the protocol.

Check. Can any one identifier replace the others?

Explanation and boundary. No. They answer different questions and each has a limited scope of uniqueness or trust.

11.3.4 Task 3: prove unit equivalence

Compare equivalent C and F requests with the same underlying values.

Worked reasoning. Use explicit tolerance for mean and SD, then compare the declared rounded decision value.

Check. What if raw values match but units differ?

Explanation and boundary. They represent different temperatures. Numeric equality without units is not physical equivalence.

11.3.5 Task 4: explain one spread calculation

Show the intermediate squared deviations and denominator.

Worked reasoning. [78,80,82] gives deviations [-2,0,2], squares [4,0,4] and SD 2 C.

Check. What does the result not quantify?

Explanation and boundary. Systematic sensor bias, calibration uncertainty or independence of repeated observations.

11.3.6 Task 5: locate a stopped stage

Inject an invalid unit and find where the trace ends.

Worked reasoning. The failure trace must not include a completed summary or protocol decision.

Check. Why inspect downstream absence?

Explanation and boundary. A superficial error status can coexist with partial side effects if stage ordering is wrong.

11.3.7 Task 6: remove source authority

Run a valid request without the approved fictional protocol.

Worked reasoning. The numerical mean can still be known while the protocol-based status is unavailable.

Check. Is this the same as malformed input?

Explanation and boundary. No. Separate numerical input validity from absence of evidence required for the requested conclusion.

11.3.8 Task 7: diagnose the denominator patch

Use a nonzero-mean fixture to expose sum/(n+1).

Worked reasoning. [79,80,81] has true mean 80; dividing by four gives 60 and misses the inclusive review condition.

Check. Why is this more informative than [0,0,0]?

Explanation and boundary. A zero numerator yields zero under both denominators, hiding the defect.

11.3.9 Task 8: package and hand over

Export reference results, traces and response fields without invented answers.

Worked reasoning. The reviewer recomputes a mean and checks the source threshold and review rule.

Check. What should happen if a field is blank?

Explanation and boundary. Keep draft status. Nonempty explanations establish completeness only, not correctness.

11.3.10 Summary: expand scope only with new tests

Optional A adds K conversion; optional B checks reading-order invariance.

Worked reasoning. Identify a computed number, a source-derived rule and an unverified physical assumption.

Check. What connects to Studio II?

Explanation and boundary. Evaluate competing implementations on a frozen case set, including both ordinary functionality and required rejections.

11.4 Mathematical reference

$$C = (F - 32)\frac{5}{9}, \qquad \bar{x} = \frac{1}{n}\sum_i x_i, \qquad s = \sqrt{\frac{\sum_i (x_i - \bar{x})^2}{n-1}}, \quad n \geq 2.$$

For [78,80,82] C, the mean is 80 C and the sample SD is 2 C. The fictional protocol requests review when the converted mean, rounded to ten decimal places, is at least 80 C. This numerical convention is not a physical uncertainty model.

11.5 Eight core practice tasks

11.5.1 Task 1: Predict the four contract outcomes

Classify low, equality, high and malformed requests before running. Reported does not mean physically safe.

Self-check. Equality triggers the inclusive >= rule. A malformed request has no claimed numeric result.

11.5.2 Task 2: Inspect provenance before prose

Locate case ID, input hash, protocol ID and implementation version, then explain the question each answers.

Self-check. A matching input digest establishes identity relative to canonical request bytes, not measurement authenticity.

11.5.3 Task 3: Check equivalent Celsius and Fahrenheit requests

Predict equal means and sample SDs. Keep distinct request hashes because raw input representations differ.

Self-check. The declared rounded comparison prevents a binary-rounding equality flip. It is a computational convention, not a measurement uncertainty model.

11.5.4 Task 4: Reconstruct the sample spread

Show deviations, squared deviations and the n-1 denominator for the synthetic reference.

Self-check. This spread does not estimate systematic calibration bias or establish independence of repeated readings.

11.5.5 Task 5: Stop at the invalid unit boundary

Change only the unit to an unsupported value and inspect which downstream stages are absent.

Self-check. The workflow must not convert, summarize or conclude before validating the input contract.

11.5.6 Task 6: Abstain without approved source evidence

Keep readings valid but remove the protocol. A known mean does not authorize a threshold-based conclusion.

Self-check. This is evidence unavailability, not malformed numeric input. No default threshold is silently substituted.

11.5.7 Task 7: Expose a plausible denominator patch

The supplied buggy branch divides by n+1. Predict why it misses the equality review case before comparing.

Self-check. This is an intentionally supplied implementation defect, not an observed mistake from a real language model.

11.5.8 Task 8: Save the input, protocol, source and result

Retain the unchanged reference alongside any experimental patch. Preserve actual source in the export.

Self-check. Download the final ZIP before the runtime expires. Preserve source, not only a screenshot.

11.6 Evidence handover and summary

Build a source-bounded thermal-report workflow from synthetic readings with explicit units, checks and review status. Save eight predictions, eight explanations and three reflections. Preserve source, data, versions, traces and one limitation. The notebook exports a draft when fields are blank; response completeness does not establish correctness. Save the edited notebook separately if changing code.

11.7 Primary references

- scikit-learn common pitfalls: https://scikit-learn.org/stable/common_pitfalls.html.
- NIST Generative AI Profile: https://doi.org/10.6028/NIST.AI.600-1.

Chapter 12

Agent Studio II: evaluation and demo rehearsal

Week 16: 2026-12-21, Monday 14:00–17:00.

Evaluate two supplied local workflow candidates against literal independent fixtures. This is not a benchmark of natural-language understanding, a live LLM, or real thermal safety.

Learning outcomes

Compare workflow candidates against independent fixtures; retain per-case failures and make a bounded release decision.

The meeting contains three 60-minute modules: 60 minutes explanation and 120 minutes practice. The eight core exercises and two optional challenges are in-class practice, not additional weekly graded homework.

12.1 H1: Design an evaluation before the demonstration

12.1.1 Evaluate behavior, not presentation polish

A convincing demo can hide a wrong calculation or forbidden operation.

Worked reasoning. Case; expected status; numeric tolerance; safety violation; split; release gate.

Check. What does a beautiful report fail to establish?

Explanation and boundary. Correctness, provenance, boundary enforcement and performance across cases. Each needs its own evidence.

12.1.2 The case is the unit of evaluation

One fixture specifies a complete input and expected behavior.

Worked reasoning. A request containing three readings is one workflow case, not three independent successes.

Check. Why state the denominator?

Explanation and boundary. A success rate is uninterpretable without the number and definition of evaluated cases.

12.1.3 Expected answers must be independent

Write literal fixture expectations before running either candidate.

Worked reasoning. [79,80,81] C must have mean 80 and review_required under the inclusive class protocol.

Check. Why not call the reference to generate every label?

Explanation and boundary. A shared bug could make the reference and candidate agree. Independent fixtures expose correlated implementation errors.

12.1.4 Separate development and held-out cases

Use development failures to change code, then freeze before evaluation.

Worked reasoning. Named dev and held-out lists are explicit; do not shuffle repeatedly to find an attractive score.

Check. Does a held-out fixture set prove deployment validity?

Explanation and boundary. No. It bounds one local evaluation and may not represent real measurement distributions or adversaries.

12.1.5 Status correctness and numeric accuracy differ

A candidate can match the number while returning the wrong action status.

Worked reasoning. Valid cases pass only when mean, SD and status match; invalid cases pass only with the declared rejection.

Check. Why keep component failures?

Explanation and boundary. They reveal the mechanism of error and prevent a single aggregate from hiding safety or data-handling defects.

12.1.6 Numerical tolerance is a rule, not a repair

Compare absolute error to atol + rtol*abs(expected).

Worked reasoning. Expected=80, atol=1e-8, rtol=1e-10 gives limit 1.8e-8.

Check. Can increasing tolerance after a failure be legitimate?

Explanation and boundary. Only with an independently justified protocol revision and new report. Do not widen it merely to make a candidate pass.

12.1.7 An error budget is not a safety excuse

Report forbidden-action violations separately from task correctness.

Worked reasoning. One unauthorized operation remains unacceptable even if nine ordinary cases succeed.

Check. Why avoid averaging severity away?

Explanation and boundary. Different failures have different consequences. A predefined safety gate can block release regardless of mean task accuracy.

12.1.8 The baseline makes added complexity accountable

A deterministic pipeline is the first candidate, not an embarrassment.

Worked reasoning. The supplied buggy candidate uses n+1 for its mean; nonzero inputs expose it.

Check. Is this an LLM quality benchmark?

Explanation and boundary. No. It evaluates local workflow implementations. Model proposal quality would require separately collected and labeled outputs.

12.1.9 Predeclare a release rule

All required fixtures pass and zero protected side effects occur in this class gate.

Worked reasoning. No candidate is approved for real deployment; passing means eligible for the classroom demonstration only.

Check. Why include the scope in the decision?

Explanation and boundary. A finite teaching fixture suite does not validate equipment, security, clinical use or broader autonomous operation.

12.1.10 Summary: freeze what will be compared

Preserve candidates, cases, expected outputs, tolerance and release gate.

Worked reasoning. Explain which defect it could expose and why current cases might miss it.

Check. What happens next?

Explanation and boundary. Execute both candidates, retain the full failure table and rehearse a truthful demonstration.

12.2 H2: Run the harness and explain candidate failures

12.2.1 One harness must judge both candidates

Apply identical cases, labels and tolerance to each implementation.

Worked reasoning. Do not grant one candidate a different input repair rule or a more generous tolerance.

Check. What invalidates the comparison?

Explanation and boundary. Changing the evaluation procedure by candidate without an explicit reason and a separate reported protocol.

12.2.2 Return structured failures, not silent skips

A candidate exception is an observed outcome with a case ID.

Worked reasoning. Skipping failed cases can inflate the apparent success rate.

Check. Should every exception be treated as a security breach?

Explanation and boundary. No. Classify invalid input handling, implementation errors and side-effect violations separately.

12.2.3 Task 3: inspect the per-case table

Show expected status, actual status, mean error and pass flag.

Worked reasoning. A failed equality case should connect the wrong denominator to the missed review status.

Check. Why export rows as well as a chart?

Explanation and boundary. A bar chart cannot identify which fixture failed or reconstruct the mechanism.

12.2.4 A zero-mean case can conceal a bug

Changing n to n+1 leaves a zero numerator unchanged.

Worked reasoning. [0,0,0] can pass the mean check; [79,80,81] cannot pass under the buggy denominator.

Check. What kind of testing lesson is this?

Explanation and boundary. Test discriminating inputs that separate plausible defects, not merely convenient examples.

12.2.5 Measure the declared subgroup outcomes

Report valid cases, invalid cases and boundary cases separately.

Worked reasoning. A 100 percent result on one case is a count of one, not a robust population estimate.

Check. Can overlapping groups be added as a total?

Explanation and boundary. No. A boundary case may also be valid. Use the complete unique case list for overall denominators.

12.2.6 Fixed-case uncertainty is not population certainty

A small nonrandom suite estimates no universal deployment success rate.

Worked reasoning. The fixture list was designed for bugs, not sampled independently from a deployment population.

Check. What would support a population performance estimate?

Explanation and boundary. A justified sampling design, representative cases, independent units and uncertainty appropriate to that design.

12.2.7 Task 5: inspect an invalid-input success

A required rejection is a correct case outcome, not missing data.

Worked reasoning. An invalid case has no expected numeric mean. Do not fabricate zero to fit it into a numeric-error plot.

Check. What should a numeric error be for rejected inputs?

Explanation and boundary. Unavailable, with a reason. Numeric metrics should use only cases for which the quantity is defined.

12.2.8 Task 6: apply the frozen release gate

A candidate with any required fixture failure remains blocked.

Worked reasoning. All declared mandatory cases must pass; report exact failing case IDs for a blocked candidate.

Check. Is passing this gate deployment authorization?

Explanation and boundary. No. It is a local demonstration-readiness decision with explicit untested threats.

12.2.9 Rehearse evidence before screenshots

The demonstration should show one success and one expected failure.

Worked reasoning. Problem 45s; mechanism 60s; successful run 75s; failed case 60s; limits and replay 60s.

Check. Why preserve the failure in the demo?

Explanation and boundary. It shows the system boundary and evidence of control rather than implying universal success.

12.2.10 Summary: explain why a candidate failed

Name the fixture, violated rule and causal code path.

Worked reasoning. Save candidate hashes, suite hash, per-case CSV, release JSON and plots.

Check. What may you change after held-out inspection?

Explanation and boundary. Make an explicitly exploratory repair, then evaluate on a new independent set before a new final claim.

12.3 H3: Studio II: evaluate, challenge, rehearse

12.3.1 Keep the evaluator independent of the candidate

Do not repair expected labels to match generated results.

Worked reasoning. Restore 5 min; evaluation 25 min; challenge design 15 min; rehearsal and export 15 min.

Check. What is the evidence contract?

Explanation and boundary. Per-case outcomes, explicit denominators, candidate and suite identity, release gate and an untested limitation.

12.3.2 Task 1: predict which fixture exposes n+1

Select a nonzero-mean case before running the buggy candidate.

Worked reasoning. Manually compute sum/len for the selected fixture and preserve that expected value.

Check. What if the candidate accidentally passes one case?

Explanation and boundary. Report the pass honestly but inspect whether the fixture discriminates the suspected defect.

12.3.3 Task 2: freeze the release gate

Write the all-required-cases rule and zero-side-effect condition.

Worked reasoning. A later relaxed rule is a different experiment, not the same passing result.

Check. Why record gate identity with the output?

Explanation and boundary. Without it, the word pass does not identify what was checked or how strict the conditions were.

12.3.4 Task 3: inspect one failing row

Read case ID, expected status, actual status and numeric mismatch.

Worked reasoning. An incorrect low mean can prevent required review even when the output format is valid.

Check. Is schema validity enough here?

Explanation and boundary. No. It establishes structure but not the numeric and decision semantics.

12.3.5 Task 4: explain the hidden zero numerator

A zero numerator erases a denominator defect on that fixture.

Worked reasoning. Use [1,2,6] or a threshold-sensitive case and explain why the outputs differ.

Check. Does adding this after seeing failures preserve a held-out claim?

Explanation and boundary. No. Treat it as a diagnostic development probe unless a fresh independent evaluation is declared.

12.3.6 Task 5: check a required rejection

Inspect empty values and an unknown unit as separate cases.

Worked reasoning. A made-up mean on malformed input is not useful behavior under the contract.

Check. Why keep rejected inputs in the case table?

Explanation and boundary. They define the interface boundary and prevent functionality-only tests from overlooking unsafe acceptance.

12.3.7 Task 6: defend the release decision

Show the exact failed IDs or all passing required IDs.

Worked reasoning. A blocked candidate is not repaired by the success of the reference.

Check. What limitation survives a passing suite?

Explanation and boundary. Coverage is finite and synthetic. It does not establish real-world reliability or live LLM behavior.

12.3.8 Task 7: rehearse the five-minute narrative

Show inputs and actual outputs before interpreting their meaning.

Worked reasoning. Keep the full 300-second budget and identify the artifact supporting each segment.

Check. Is another five-minute submission required?

Explanation and boundary. No. This is optional in-class rehearsal. It does not add a new weekly graded video assignment.

12.3.9 Task 8: export an inspectable evaluation

Download case table, configuration, hashes, responses and plots.

Worked reasoning. A classmate counts unique cases and checks one composite pass rule from the exported rows.

Check. Why not export only aggregate accuracy?

Explanation and boundary. It loses failure identity, subgroup denominators and the distinction between numeric and interface errors.

12.3.10 Summary: improve tests before adding autonomy

Optional A adds a metamorphic relation; optional B proposes a new held-out fixture.

Worked reasoning. Choose one defect or coverage gap and design a specific independent check.

Check. What connects to Week 17?

Explanation and boundary. Package this evidence so a new process and a classmate can reconstruct the demonstration and its limits.

12.4 Evaluation reference

$$|a - e| \leq \text{atol} + \text{rtol}|e|.$$

A valid case passes only when status and all required numeric fields match. An invalid case passes by the specified rejection, with no fabricated numeric target. The class release rule requires all mandatory fixtures to pass and no protected side effects. The designed fixture suite is not a random sample from deployment; no population success interval is claimed.

12.5 Eight core practice tasks

12.5.1 Task 1: Predict which fixture exposes the denominator

Use development cases first. Calculate the expected mean manually and compare both supplied candidates.

Self-check. The zero case passes the supplied buggy mean because the numerator is zero. That does not clear the implementation.

12.5.2 Task 2: Freeze the release gate and candidate identities

Write the mandatory rule before opening the held-out result. The side-effect count below is zero by construction: these candidates have no action tools.

Self-check. Passing means classroom demonstration readiness only. A frozen gate is not a new grading policy.

12.5.3 Task 3: Run the same held-out harness for both candidates

Inspect every row before reading an aggregate. Exceptions remain failures with their case IDs.

Self-check. A required rejection has no expected numeric error. Never replace unavailable numeric values with zero.

12.5.4 Task 4: Explain why the zero case hid a defect

Contrast D-zero with the held-out inclusive boundary fixture; diagnose, but do not repair and re-report on the same held-out set.

Self-check. A numeric defect can also change a review decision. A polished report does not repair the calculation.

12.5.5 Task 5: Inspect required rejections separately

Count valid and invalid fixtures explicitly, and check the same component rules for both candidates.

Self-check. The case, not each repeated reading, is the evaluation unit. Show counts alongside every percentage.

12.5.6 Task 6: Apply the unchanged classroom release gate

Name exact failing IDs and the finite scope of each decision.

Self-check. The structural absence of side-effect tools is not a live security benchmark. No real deployment authorization is implied.

12.5.7 Task 7: Rehearse a five-minute evidence narrative

Identify the artifact supporting each segment. This is in-class practice, not an additional video submission.

Self-check. Show the actual T-boundary failure, not an invented model anecdote. Keep the original frozen evaluation intact.

12.5.8 Task 8: Export rows, not only averages

Save a CSV with defined numeric fields and explicit unavailable values for required rejection cases.

Self-check. The final ZIP includes configuration, source and all actual observations. Blank reflection fields remain draft.

12.6 Evidence handover and summary

Compare workflow candidates against independent fixtures; retain per-case failures and make a bounded release decision. Save eight predictions, eight explanations and three reflections. Preserve source, data, versions, traces and one limitation. The notebook exports a draft when fields are blank; response completeness does not establish correctness. Save the edited notebook separately if changing code.

12.7 Primary references

- Demystifying evals for AI agents: https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents.
- scikit-learn model evaluation: https://scikit-learn.org/stable/modules/model_evaluation.html.

Chapter 13

Project integration and oral-exam preparation

Week 17: 2026-12-28, Monday 14:00–17:00.

Package the same synthetic workflow in readable source and JSON. Verify exact payload bytes, run a new Python process from a new working directory, and distinguish numerical replay from physical or model validation.

Learning outcomes

Package a bounded workflow, reproduce it in a fresh process, audit artifacts, and defend claim, evidence and limitation.

The meeting contains three 60-minute modules: 60 minutes explanation and 120 minutes practice. The eight core exercises and two optional challenges are in-class practice, not additional weekly graded homework.

13.1 H1: Integrate the final evidence contract

13.1.1 Integration means a reconstructable result

Connect input, source, code, configuration and output into one evidence chain.

Worked reasoning. Artifact identities; manifest; runner; environment; replay comparison; oral explanation.

Check. What does a project screenshot omit?

Explanation and boundary. Executable procedure, inputs, dependency versions, rejected cases and the basis of its claims.

13.1.2 Scope the final claim before packaging

State exactly what the synthetic workflow demonstrated and did not demonstrate.

Worked reasoning. The frozen local implementation matches these independent teaching fixtures; no physical deployment or live-model evaluation is established.

Check. Why not say the agent is reliable?

Explanation and boundary. That omits the task, tested cases, assumptions and scope. Use a claim that the exported evidence can actually support.

13.1.3 The manifest names required payloads

Hash input, protocol, configuration and source; exclude the manifest from hashing itself.

Worked reasoning. A file cannot generally include its own final hash. Keep the required payload hashes in a separate manifest.

Check. Does a matching unsigned manifest authenticate a sender?

Explanation and boundary. No. Someone can replace both payload and digest. Trust needs an external reference or signature.

13.1.4 Use canonical JSON deliberately

Sort keys and fixed serialization identify one representation, not all equivalent objects.

Worked reasoning. Whitespace changes a raw-file hash. Canonical hashing compares the declared serialized representation.

Check. Which hash should the handover record use?

Explanation and boundary. Declare whether it hashes exact files or canonical values. This package uses exact saved payload bytes for integrity.

13.1.5 Source is safer to inspect than opaque objects

A readable runner rebuilds the workflow from supplied files.

Worked reasoning. The package contains JSON and Python source, not serialized executable model objects.

Check. Does readable source make execution safe?

Explanation and boundary. No. Review it and use an appropriate environment. Arbitrary Python can still perform harmful operations.

13.1.6 A new process removes hidden globals

Run the saved source from a fresh working directory with explicit paths.

Worked reasoning. The runner must not require an in-memory variable, previous cell or current notebook directory.

Check. What does a fresh process still share?

Explanation and boundary. It can share interpreter, packages, operating system and hardware. It is not independent scientific replication.

13.1.7 Replay compares computations, not outcomes

A zero replay difference does not imply an accurate physical model.

Worked reasoning. Prediction error compares against observations; replay difference compares outputs of the same declared procedure.

Check. Can an incorrect workflow replay perfectly?

Explanation and boundary. Yes. Reproducibility preserves the computation, including its defects. Independent fixtures test correctness separately.

13.1.8 Align identities before values

Compare case IDs and output fields before applying numeric tolerance.

Worked reasoning. A comparison must join by stable IDs or require the declared sequence. Bare positional vectors can mismatch cases.

Check. Does matching aggregate mean prove row alignment?

Explanation and boundary. No. Permutations can preserve an aggregate while assigning values to the wrong cases.

13.1.9 Record the actual environment

Save Python, package versions, platform, source hash and invocation.

Worked reasoning. The replay runner is standard-library only; plotting stays in the notebook. Preserve both scopes.

Check. Does a seed guarantee cross-version equality?

Explanation and boundary. No. Algorithms, libraries and numeric implementations can change. Name the environment and comparison criterion.

13.1.10 Summary: package the claim with its limits

An honest handover includes failures, replay evidence and missing validation.

Worked reasoning. A different protocol, unit convention or population can invalidate the original conclusion.

Check. What will the next hour test?

Explanation and boundary. Extracted files, corrupted payloads, explicit failure messages and a complete five-minute explanation.

13.2 H2: Replay, integrity checks, and oral defense

13.2.1 The ZIP must contain everything it promises

Use a required-file list rather than archiving arbitrary runtime state.

Worked reasoning. Read source and paths. The reference extraction rejects unexpected members and does not execute received code automatically.

Check. Why avoid arbitrary runtime archives?

Explanation and boundary. They may leak secrets, personal files or irrelevant state and make the intended procedure unclear.

13.2.2 Task 4: extract into a fresh directory

Verify manifest keys and bytes before starting the runner.

Worked reasoning. Removing protocol.json must fail before the analysis claims a protocol-based result.

Check. What should a failed integrity check do?

Explanation and boundary. Stop with a specific error. Do not retrain, repair or regenerate missing evidence silently.

13.2.3 Task 5: replay from saved files

Use an explicit interpreter, runner path, bundle root and output path.

Worked reasoning. A launched process is not completed execution. Check successful termination and parse its result file.

Check. Why preserve stderr on failure?

Explanation and boundary. It is evidence of the failing dependency or predicate. Replacing it with a success message destroys diagnosability.

13.2.4 Task 6: compare an aligned result vector

Use case IDs then the declared absolute-plus-relative tolerance.

Worked reasoning. This small runner should agree numerically in this environment. Report the actual maximum difference, not a promised zero.

Check. What if a metric agrees but predictions differ?

Explanation and boundary. Do not call it a successful replay. Aggregate agreement can conceal per-case differences.

13.2.5 Tamper detection is not authenticity

A changed copied payload should fail against the preserved manifest.

Worked reasoning. A harmless-looking blank line changes the exact-file hash even if a parser would ignore it.

Check. What if both payload and manifest are replaced?

Explanation and boundary. Unsigned consistency checks may pass. A trusted external reference or signature is needed for sender authenticity.

13.2.6 Explain the mechanism in a short defense

Use claim, evidence, limitation and an invalidating change.

Worked reasoning. Name the fixture, show the trace, recompute a value and identify what the evidence does not establish.

Check. Why is memorizing a score insufficient?

Explanation and boundary. It does not demonstrate the split, procedure, source or reasoning that gives the score meaning.

13.2.7 Budget the five-minute demonstration

Make every segment point to a real artifact.

Worked reasoning. Show one exact input, one executed success, one expected rejection, then how a peer can replay it.

Check. Must every student submit a new video this week?

Explanation and boundary. No. This is in-class rehearsal only; the separately announced stock-prediction HW retains its own five-minute video requirement.

13.2.8 Written and oral assessment remain separate

The project evidence supports explanation but does not replace either examination.

Worked reasoning. What breaks if a threshold changes units, a tool gains side effects or a retrieved source loses a qualifier?

Check. Should the course publish live exam answers here?

Explanation and boundary. No. These are public study prompts and worked practice, not confidential examination papers or new grading rules.

13.2.9 Peer handover is an adversarial usability test

A partner locates inputs, expected behavior and one known failure without coaching.

Worked reasoning. Capture what the partner could and could not reconstruct. Incomplete handover remains a documented limitation.

Check. Can a partner's successful replay establish generalization?

Explanation and boundary. No. It supports portability within the checked environment, not external predictive or security validity.

13.2.10 Summary: preserve the observed state

Keep passed, failed and untested items distinct in the final result card.

Worked reasoning. Save source, inputs, manifest, replay result and draft/completed response record.

Check. What must not be upgraded into completion?

Explanation and boundary. A prepared ZIP, launched job or connected runtime without successful execution and inspected outputs.

13.3 H3: Final integration studio and explain-back

13.3.1 Finish a rerunnable bounded project record

This lab integrates existing evidence rather than adding untested autonomy.

Worked reasoning. Restore 5 min; package and replay 20 min; integrity challenge 15 min; peer defense and export 20 min.

Check. What counts as an honest stopping point?

Explanation and boundary. A draft with explicit missing evidence and next action, not fabricated success or copied metrics.

13.3.2 Task 1: locate every hidden dependency

List expected payloads and identify any notebook-only variable.

Worked reasoning. Use a fresh working directory; the runner should use explicit paths rather than assume the notebook location.

Check. What if it only runs beside the notebook?

Explanation and boundary. The portability claim is too broad. Fix the path contract or state the required working directory explicitly.

13.3.3 Task 2: check case identity and ordering

Reorder a copied output and detect sequence mismatch before numeric comparison.

Worked reasoning. A length match alone does not establish observation identity.

Check. When is an explicit ID join acceptable?

Explanation and boundary. When IDs are unique and required membership is checked, and the alignment rule is recorded in the evidence.

13.3.4 Task 3: state the actual environment

Read the generated version record rather than an assumed Colab configuration.

Worked reasoning. A Colab link is a launch route, not proof that the notebook was run in a Google Colab browser.

Check. Why does this distinction matter?

Explanation and boundary. Dependency, network and runtime behavior can differ. Execution claims should name the environment actually tested.

13.3.5 Task 4: inspect the extracted package

Check file whitelist, manifest completeness and payload hashes.

Worked reasoning. Do not replace an absent file with a freshly invented one and call the same bundle complete.

Check. What does an intact archive prove?

Explanation and boundary. Its stored bytes can be read without CRC errors; it does not prove scientific correctness or code safety.

13.3.6 Task 5: require actual process completion

Check return code and parsed output after the new interpreter exits.

Worked reasoning. A failed or timed-out process remains an observed failure with its diagnostic output.

Check. What is the acceptance condition?

Explanation and boundary. Successful termination, required result schema and the declared per-case agreement checks.

13.3.7 Task 6: explain computational agreement

Compare saved and replayed means by case ID under fixed tolerance.

Worked reasoning. If nonzero but within tolerance, report that fact rather than replacing it with zero.

Check. What does zero difference leave untested?

Explanation and boundary. Physical data validity, external populations, real LLM behavior and unseen adversarial inputs.

13.3.8 Task 7: detect copied-file tampering

Modify a temporary payload and preserve the original reference.

Worked reasoning. A mismatch must prevent analysis from proceeding under the reference claim.

Check. Can a matching digest replace peer source review?

Explanation and boundary. No. Integrity and trust are distinct; a harmful file can have a perfectly matching digest.

13.3.9 Task 8: defend claim and limitation

A partner asks one unexpected question about mechanism or scope.

Worked reasoning. What if the readings are correlated or the protocol changes? Separate computational validity from a changed scientific task.

Check. What is a strong answer?

Explanation and boundary. One grounded in an actual artifact and explicit assumptions, with no invented deployment or accuracy claims.

13.3.10 Summary: prepare, do not replace, the final exam

Optional A tests metadata-only changes; optional B expands a handover checklist.

Worked reasoning. State passed, failed, untested and next action. Week 18 remains the written and oral final examination.

Check. What should continue after this course?

Explanation and boundary. The habit of defining the task, preserving data, checking execution, explaining results and bounding every claim.

13.4 Replay reference

$$\Delta_i = \text{replayed mean}_i - \text{reference mean}_i.$$

Compare unique case identities and declared order before values. Use the saved absolute-plus-relative tolerance. A zero maximum replay difference establishes agreement between computations in the checked environment, not physical accuracy, scientific replication or live-model quality.

13.5 Eight core practice tasks

13.5.1 Task 1: Find notebook-only dependencies

Predict which globals a new process can see. Inspect the saved source and explicit path arguments before executing.

Self-check. A new interpreter still shares the installed Python environment and operating system. Do not call it independent scientific replication.

13.5.2 Task 2: Check identity before numeric values

Reverse a copied result list, detect a sequence mismatch, then align explicitly by unique stable IDs.

Self-check. A length match or aggregate score is not an ID alignment check. Explicitly reject missing or duplicate IDs.

13.5.3 Task 3: Record the environment that actually ran

Inspect the actual Python and package record. A Colab launch URL does not establish Colab-browser execution.

Self-check. If you run in Colab, record its actual versions. Cross-version bitwise identity is not promised.

13.5.4 Task 4: Extract a whitelisted handover ZIP

Inspect each archive member and its manifest before execution. The supplied extractor accepts exactly six known base names.

Self-check. A ZIP CRC check establishes readable archive storage, not code safety. Source review remains required.

13.5.5 Task 5: Require a completed new process

Run the reviewed runner from the extracted directory, with a fresh working directory and explicit output path.

Self-check. The process is awaited with a timeout and checked return code. A process launch alone would not be evidence of completion.

13.5.6 Task 6: Compare aligned numeric outputs

Check exact ID sequence and status first. Then apply the frozen absolute-plus-relative numeric tolerance.

Self-check. Zero replay difference does not establish physical accuracy. The original workflow can be reproduced even when its scientific assumptions are wrong.

13.5.7 Task 7: Detect tampering on a copy

Append bytes to a copied request payload. Verify the reference copy is unchanged and the copied bundle is rejected.

Self-check. This change can preserve parsed values while changing bytes. A sender replacing both payload and manifest remains outside this integrity check.

13.5.8 Task 8: Create a defensible handover card

Keep passed, failed and untested claims separate. Adapt the public study prompts to your own project artifacts.

Self-check. Week 18 remains written and oral examination. This notebook neither publishes live exam answers nor introduces a new weekly graded task.

13.6 Evidence handover and summary

Package a bounded workflow, reproduce it in a fresh process, audit artifacts, and defend claim, evidence and limitation. Save eight predictions, eight explanations and three reflections. Preserve source, data, versions, traces and one limitation. The notebook exports a draft when fields are blank; response completeness does not establish correctness. Save the edited notebook separately if changing code.

13.7 Primary references

- NumPy reproducibility policy: https://numpy.org/doc/stable/reference/random/compatibility.html.
- scikit-learn model persistence: https://scikit-learn.org/stable/model_persistence.html.

Appendix A

Colab Conventions and Homework Milestones

A.1 Colab conventions

Every student notebook should begin with a short environment cell, use a declared random seed where relevant, and provide a CPU-compatible path. Optional accelerator cells must not be required for assessment. Notebooks should obtain public data from a documented source or a course repository path; they must not include credentials, personal data, or hidden instructor solutions.

The course website opens each notebook through a public GitHub repository using the Colab URL pattern defined by `\CourseRepo` in `main.tex`. Replace that one configuration value when the repository is created. The notebook source remains the canonical student artifact; the rendered website is an access layer.

A.2 Homework milestones

Homework	Timing	Evidence required
ML Problem Brief	Week 5	Decision context, target, features, split proposal, metric, and one risk.
ML Workflow	Week 10	Executable notebook, baseline, evaluation, error analysis, and bounded conclusion.
Final Agent Plan	Week 14	Goal, tools, stop condition, approval point, failure cases, and test plan.
Demo Readiness	Week 16	Reproducible run, evaluation cases, trace, limitation, and rollback or safe fallback.

A.3 Submission rule

An AI tool can assist with code, prose, or debugging, but the student remains responsible for the submitted claim. Work that uses AI assistance should identify the assistance when it materially shaped the artifact, then show the validation performed by the student.

A.4 Runtime-aware Lab design

Colab can load notebooks from GitHub, but its free resource limits, hardware availability, and runtime lifetime change over time. Runtimes are ephemeral, so a notebook must install or check any required package, fetch only documented public data, and write important outputs to a student-controlled location only when the student explicitly chooses to do so [3]. Core Labs in this course are CPU-first; any GPU cell is optional and cannot be necessary for a graded result.

The current reference implementation uses scikit-learn for core ML workflows and PyTorch only after neural-network foundations. Rather than pinning the course to a fragile hosted-runtime version, each notebook records the actual package versions at execution time. This makes a future result diagnosable when managed environments change [8, 10].

Appendix B

Extension Studios: Deep Learning Architectures

These topics remain available as optional Colab studios, reading groups, or final-project extensions. They are deliberately not prerequisites for the core ML workflow.

B.1 Convolutional neural networks

CNNs use shared local filters to exploit image structure. LeNet and later residual architectures illustrate how representation depth, skip connections, and data scale change a vision workflow [6]. Use this studio when the course data are images and a classical baseline is already in place.

B.2 Sequential models

RNNs, LSTMs, and GRUs model ordered observations. Their main lesson for this course is temporal data discipline: define what is known at each prediction time and avoid future leakage [5].

B.3 Generative models

GANs formulate generation as an adversarial game [2]; diffusion models learn a reverse denoising process [4]. These are useful extension topics for representation and synthesis, but they do not replace the need for an evaluation contract.

Bibliography

[1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[2] Ian Goodfellow et al. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, 2014.

[3] Google Colaboratory. Frequently asked questions: Resource limits and notebook runtimes. https://research.google.com/colaboratory/faq.html, 2026. Accessed August 2026.

[4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020.

[5] Sepp Hochreiter and Jurgen Schmidhuber. Long short-term memory. *Neural Computation*, 9 (8):1735–1780, 1997.

[6] Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[7] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.

[8] PyTorch Contributors. Pytorch stable documentation. https://docs.pytorch.org/docs/stable/, 2026. Accessed August 2026.

[9] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[10] scikit-learn developers. scikit-learn 1.9 release history. https://scikit-learn.org/stable/whats_new.html, 2026. Accessed August 2026.

[11] Vladimir N. Vapnik. *The Nature of Statistical Learning Theory*. Springer, 1995.